



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A CoMove szoftveralkalmazás dokumentációja

Készítették:

DARUNDAY MARIAH LLIANNE

MEGYERI BENCE

GEOSITS ÁRON ANDRÁS

SZOMBATHELY

2026

Tartalomjegyzék

Bevezetés.....	4
Mi az a CoMove?	4
Témaválasztás indoklása.....	4
Hasonló szolgáltatások.....	5
Célközönség	5
A szoftver műszaki feltételei és futtatása.....	6
Adatbázis.....	6
Backend.....	6
Frontend	7
Fejlesztői környezet és a szoftver közzététele (deployment).....	8
Frontend	8
Backend és Adatbázis	8
Windows	8
MacOS.....	8
Kollaboráció és közzététel (deployment).....	8
Komponensek technikai leírása.....	9
Adatbázis.....	9
Egyed típusok.....	9
Felhasználó (users).....	9
Token (usertokens).....	11
Értesítés (notifications)	11
Jármű (vehicles)	12
Jármű elérhetőség (vehicleavailabilities)	13
Járműkép (vehicleimages).....	13
Bérlés (rentals)	14
Üzenet (messages).....	15

Kapcsolatok.....	16
Implementáció.....	17
Backend.....	18
Metóduskontrollerek	18
Autentikációs kontroller – AuthController	18
Felhasználó kontroller – UserController	21
Bérlés kontroller – RentalController	25
Jármű kontroller – VehicleController	29
Adminisztrátor Kontroller – AdminController.cs	38
Szolgáltatások	39
Autentikációs szolgáltatás – AuthService	39
Email szolgáltatás – IEmailService – ResendEmailService	40
Képerőforrás-kezelő szolgáltatás – IResourceService – LocalResourceService	40
Bérléskezelő szolgáltatás – RentalService	41
Válaszszűrőrendszer – VisibilityFiltering	43
Frontend	44
Oldalak	44
Komponensek.....	44
EnsureAuth.....	44
Navbar és NotificationMenu	44
CenteredLayout és PageLayout.....	44
Scriptek	45
Reszponzivitás.....	45
A frontend felület használatának bemutatása.....	46
Főoldal.....	46
Jármű kiválasztása.....	47
Regisztráció és Bejelentkezés	48

Bérlés folyamata.....	49
Profil.....	51
Beállítások.....	51
Bérléseim.....	52
Járműveim	52
Adminisztrátori felület	53
Kijelentkezés	53
Továbbfejlesztési lehetőségek.....	54
Teszt dokumentáció	55
TestHandler	55
Kontroller metódusok tesztelése	55
Stressztesztelés	56
Mellékletek.....	57
ER-modell	57
Relációs diagramm.....	58

Bevezetés

Mi az a CoMove?

Az alábbi dokumentáció a CoMove elnevezésű projektmunkánkat mutatja be, amely egy közösségi autómegosztó platform. A CoMove egy olyan webes felület, amely közvetlen kapcsolatot teremt a magángépjármű tulajdonosok és az azokat bérelni szándékozó felhasználók között, lehetővé téve az autók biztonságos és egyszerű bérbeadását magánszemélyek számára.

Témaválasztás indoklása

A 2024-es tanév végén tudtuk, hogy a következő évben egy olyan összetett programot kell alkotnunk és azt prezentálni, amely, ha létezik is, ilyen formátumba nem található meg a piacon. Így a projekt témájának kiválasztásakor próbáltunk olyan területet keresni, amely egyszerre jelent technikai kihívást és választ ad egy valós társadalmi problémára. A közösségi autómegosztás mellett több meghatározó érv is szólt.

Nap mint nap láthatjuk, hogy a környezetünkben rengeteg autó áll kihasználatlanul, miközben a saját jármű fenntartása egyre drágább. Azt tapasztaltuk, hogy bár léteznek autókölcsönzők, nincs egy olyan platform, amely kifejezetten a magánszemélyek közötti autócserére specializálna. Olyan rendszert akartunk fejleszteni, ami erre a problémára megoldást nyújthat. Fejlesztői szempontból ez a téma több összetett területet is érint:

- Meg kellett oldanunk a valós idejű kommunikációt a felek közt,
- a bérleti tranzakciók lebonyolítását,
- és egy olyan véleményező rendszernek a létrehozását, amellyel a felhasználók megbízhatnak egymásban.

Hasonló szolgáltatások

A projekt tervezése során megvizsgáltuk a jelenlegi piacot. A legtöbb nemzetközi és hazai járműveket bérbeadó cég (pl. Avis, Hertz, Sixt) saját autóflottával dolgozik, ami magas bérleti díjakat, meghatározott választékot, és személytelen ügyintézkést eredményez, illetve a problémát csak a bérlő oldaláról próbálja megoldani.

Miben kínál mást a CoMove?

A legtöbb hasonló szolgáltatással szemben mi nem egy újabb autókölcsönző céget akartunk létrehozni, hanem egy közösségi felületet. A fejlesztés során két ponton tettük egyedivé a platformunkat:

- **A személyesség és a közvetlen kapcsolat:** Hisz míg a nagyobb szolgáltatóknál az ügyfél egy anonim bérlő, nálunk a beépített üzenőfelület segítségével a felek kapcsolatba léphetnek. Ez lehetőséget ad a részletek rugalmas megbeszélésére, szükség szerinti változtatások egyeztetésére.
- **Biztonságos és informált bérlet:** A platformon nem csak az autót lehet véleményezni, hanem a bérlőt is, így a tulajdonosok nyugodt szívvel adhatják át a kulcsot egy olyan fiatalnak is, akinek még nincs saját autója, de a visszajelzései alapján megbízható.
- **Bérbeadás:** A szolgáltatás nem csupán bérletre ad lehetőséget, hanem lehetővé teszi a nem használt járművek bérbeadását is, így passzív jövedelemhez juttatva annak tulajdonosát.

Célközönség

A szoftver célközönsége tágran meghatározható:

- **Bérbeadó** szerepben hasznos lehet járművet keveset használó személyeknek (pl.: csak hétvégén), vagy például valakinek, aki huzamosabb időre külföldre utazik.
- **Bérlő** szerepben még nagyobb lehet a felhasználótábor. Rengeteg személy él nagyvárosokban, akiknek egy autó fenntartása nem érné meg a mindennapokban, viszont alkalmanként szüksége lenne valamilyen járműre egy hosszabb út megtételéhez.

A szoftver műszaki feltételei és futtatása

Adatbázis

Az adatbázis **MySQL** alapú, így, hogy hozzáférjük ahhoz, szükséges egy MySQL szerver telepítése, majd az adatbázis-dump beimportálása. Mi ehhez **XAMPP**-ot használtunk, mivel segítségével a MySQL szerver könnyen indítható, az adatbázis-dump (a forráskódban a *comove.sql* fájl) pedig könnyen importálható phpMyAdmin-ban. Fontos, hogy telepítés után a MySQL szerver a **3306**-os porton fut, azonban a backend azt a **3307**-es porton várja, így vagy a MySQL konfigurációját változtatjuk meg (a XAMPP mappájában, a *my.ini* fájlban írjuk át a portot), vagy a backend ConnectionString-jét (az *appsettings.json*-ben írjuk át a portot).

Az adatbázis-dump tartalmaz példa járműveket, bérlesek és felhasználókat, az ő bejelentkezési információik:

- E-mail: tesztelek@teszt.hu; Jelszó: NagyTesztElek32
- E-mail: gipszjakab@teszt.hu; Jelszó: KisGipszJakab64
- E-mail: vincseszter@teszt.hu; Jelszó: NagyVincsEszter128
- Adminisztrátor - E-mail: admin@comove.app; Jelszó: NagyAdmin123

Backend

A szoftver backend része ASP.NET Core-on alapul, .NET 8-as verzió és C#-ban íródott. Ez legkönnyebben Visual Studio-ból futtatható. A projekt építése (build) során ez elméletileg letölti a szükséges NuGet könyvtárakat/csomagokat, mint a *MySQL.EntityFrameworkCore*, a *FileTypeChecker*, vagy a *Resend*. A projekt **https**-en a **7245**-ös porton fut, **http**-n pedig az **5126**-oson. Első **https**-sel történő futtatáskor el kell fogadni a felugró ablakokat, amelyek szerint SSL tanúsítványokat fogadtat el a számítógépünkkel a Visual Studio, másképpen nem tudnánk **https**-sel futtatni a backendet. Amennyiben nem a **https** indítási konfigurációt, hanem a **http**-t használjuk, a *HttpOnly* cookie beállításai megváltoznak, hogy továbbra is funkcionálisak maradjanak.

Az e-mail küldő funkcionalitás a **Resend**¹ API-val működik, ehhez szükség van egy API kulcsra, amit az *Auth:Mail:Resend:Token* alatt kell tárolni az *appsettings*-ben (vagy a *.NET User Secrets*-ben vagy egy környezeti változóban), másképpen a funkció nem elérhető. Ez az élő oldalon² tesztelhető. Az API kulcs itt érhető el, mivel nem biztonságos azt feltölteni

¹ <https://resend.com/>

² <https://comove.app/>

forráskódba (valójában ide sem, így meglehet, hogy a következő token, mikor ezt olvassa, már nem él): `re_M9PygXoe_GiYn8TvgZAvuUNWJ65kGR2Dr`

Frontend

A frontend futtatásához/építéséhez (build) Node.js 24.15.0-ás verzió szükséges npm-mel. A CoMove mappában adjuk ki az `npm install` parancsot, majd várjuk meg amíg feltelepül a **Vite** és a további frontendhez szükséges csomagok:

- a **Mantine**, amely egy komponenskönyvtár React-hez,
- a **Tanstack Query**, amely egy globális állapotmenedzselő könyvtár, amely a válaszokat cache-eli és periodikusan automatikusan lekéri a backend-től,
- a **React Router**, amely a komponensek oldalakra osztását, és az azok közti navigálást végzi, vagy
- az ikoncsomagok, mint az `tabler/icons-react` vagy a `react-icons`.

Miután a csomagok sikeresen települtek az `npm run dev` parancs kiadásával tudjuk elindítani a frontendet fejlesztői módban. Amennyiben statikus fájlakká szeretnénk exportálni az `npm run build` parancsot kell kiadnunk. Fontos, hogy a frontend alapvetően **https**-en és **localhost**-on próbál kapcsolatba lépni a backend-el. Ezt a `Config.js`-ben átírhatjuk, vagy megadhatunk másik backend útvonalat egy környezeti változó (a `VITE_BACKEND_URL`) segítségével.

Fejlesztői környezet és a szoftver közzététele (deployment)

Frontend

A frontend fejlesztéséhez React-et használtunk Vite-tal, Mantine komponenskönyvtárral, Tanstack Query globális React állapottároló könyvtárral, és React Router-rel. A kódot Visual Studio Code-ban írtuk.

A frontend megjelenítéséhez bármilyen modern böngésző használható. Mi a Google Chrome-ot használtuk, főleg mivel a vendég profilok segítségével egyszerűen tudtuk tesztelni a bérlő és bérbeadó szerepéből párhuzamosan a bérlési rendszert.

Backend és Adatbázis

Windows

Az adatbázishoz XAMPP-ot használtunk MySQL-lel.

Az ASP.NET backend fejlesztéséhez Visual Studio Community 2022-t használtunk.

MacOS

Az adatbázishoz DBngin-t használtunk MySQL-lel, valamint az adatbázishoz való közvetlen hozzáféréshez JetBrains DataGrip-et.

Az ASP.NET backend fejlesztéséhez JetBrains Rider-t használtunk.

Kollaboráció és közzététel (deployment)

A kódot elsősorban egy közös **GitHub repositoryban** osztottuk meg, ebben dolgoztunk mindannyian. A **GitHub Actions** segítségével egy Continuous Integration (CI) és Continuous Deployment (CD) folyamatot hoztunk létre, amely a main branch-re/ágra feltöltött kódot automatikusan megpróbálja build-elni, valamint tesztelni a megírt egységtesztek segítségével. Amennyiben ezek sikeresek voltak, a build-elt kódot átmásolja egy távoli szerverre, amelyet a <https://comove.app> domainen el is lehet érni, így a szoftvert valós körülmények alatt is ki lehet próbálni. A domaint és a szerverhasználatot a GitHub Student Developer Pack³ segítségével tudtuk ingyen megszerezni.

³ <https://education.github.com/pack>

Komponensek technikai leírása

A szoftverprojekt alapvetően három részre oszlik fel: az adatbázisra, a backendre, és a frontend weboldalra.

Adatbázis

Az adatbázis felépítéséről készítettünk ER-modell (ábra 22)-t, majd leképezés után egy Relációs diagramm (ábra 23)-ot.

Egyed típusok

A szoftveralkalmazás igényei szerint az alábbi egyed típusokat és táblákat határoztuk meg:

Felhasználó (users)

A felhasználó adatait tároló egyed típus. A redundancia elkerülése érdekében nem szedtük külön egyed típusokra a bérbeadókat és bérlőket, hanem egy felhasználó mindkét szerepben részt vehet bérletekben.

Attribútumai/mezői:

- **id:** A felhasználó rendszerben használt azonosítószáma, a létrejövő tábla elsődleges kulcsa, így minden felhasználónak egyedi. Típusa: szám (int), nem lehet null.
- **idCardNumber:** A felhasználó személyi igazolványának száma, alapvetően adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), nem lehet null.
- **name:** A felhasználó neve. Típusa: szöveg (string / varchar), nem lehet null.
- **phone:** A felhasználó telefonszáma, adminisztrációs célokból tárolva. Típusa: szöveg (string / varchar), nem lehet null.
- **dateOfBirth:** A felhasználó születési dátuma, megerősítésként, hogy elmúlt 18 éves, illetve adminisztrációs okokból tárolva. Típusa: dátum (DateOnly / date), nem lehet null.
- **profilePicPath:** A felhasználó profilképéhez vezető elérési út. Típusa: szöveg (string / varchar), lehet null.
- **email:** A felhasználó e-mail címe, bejelentkezési és üzenetküldési célokból tárolva. Típusa: szöveg (string / varchar), nem lehet null.
- **password:** A felhasználó jelszavának hash-je. Típusa: adathalmaz (byte[] / blob), nem lehet null.
- **salt:** A felhasználó jelszavának hash-eléséhez használt só/salt. Típusa: adathalmaz (byte[] / blob), nem lehet null.
- **role:** A felhasználó szerepe a rendszerben, lehet felhasználó vagy adminisztrátor. Típusa: enum, nem lehet null.

- **driversLicenseNumber:** A felhasználó vezetői engedélyének száma, adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), lehet null, hiszen valaki jogosítvány nélkül is birtokolhat és feltölthet járművet.
- **Cím:** A felhasználó lakcíme összetett attribútum, így több alattribútumra oszlik fel:
 - **addressZipcode:** A felhasználó címének irányítószáma, adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), nem lehet null.
 - **addressSettlement:** A felhasználó lakhelyének települése, adminisztrációs okokból tárolva, illetve a járműveinek településekhez rendeléséhez. Típusa: szöveg (string / varchar), nem lehet null.
 - **addressStreetHouse:** A felhasználó címének utca, házszám része, adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), nem lehet null.
- **balance:** A felhasználó egyenlege, amelyből bérelhet járművet, illetve bérbeadói szerepben, amelyre megkapja a bérlet díját. Típusa: szám (int), nem lehet null, alapvető értéke 0.

Token (*usertokens*)

A jelszó visszaállításához szükséges kódokat tartalmazó egyedtípus/tábla.

Attribútumai/mezői:

- **id:** A rendszerben a tokent azonosító szám, a létrejövő táblában elsődleges kulcs. Típusa: szám (int), nem lehet null.
- **token:** Az visszaállító/megerősítő kód. Típusa: szöveg (string / varchar), nem lehet null.
- **type:** A kód típusa, lehet jelszóvisszaállító, illetve a rendszer bővíthető lehet e-mail cím megerősítéssel. Típusa: enum (int), nem lehet null.
- **timeCreated:** Az időpont, amikor a token generálva lett. Típusa: időpont (datetime, UTC), nem lehet null

A leképezés után létrejövő idegen kulcs(ok):

- **userId:** A kódhoz tartozó felhasználó azonosítószáma, idegen kulcs. Típusa: szám (int), nem lehet null. A *users* táblára mutat.

Értesítés (*notifications*)

A felhasználónak küldött rendszerértesítéseket tartalmazó egyedtípus/tábla.

Attribútumai/mezői:

- **id:** A backenden és az adatbázisban az értesítést azonosító szám, a létrejövő táblában elsődleges kulcs. Típusa: szám (int), nem lehet null.
- **notificationId:** A felhasználó kontextusában az értesítést azonosító szám. Típusa: szám (int), nem lehet null.
- **content:** Az értesítés szövege. Típusa: szöveg (string / varchar), nem lehet null.
- **timeSent:** Az időpont, amikor a rendszer az értesítést kiküldte. Típusa: időpont (datetime, UTC), nem lehet null.

A leképezés után létrejövő idegen kulcs(ok):

- **userId:** A felhasználó azonosítószáma, akit az értesítés céloz, idegen kulcs. Típusa: szám (int), nem lehet null. A *users* táblára mutat.

Jármű (vehicles)

Az oldalra feltöltött jármű adatait tároló egyedtípus.

Attribútumai/mezői:

- **id:** A jármű rendszerben használt azonosítószáma, a létrejövő tábla elsődleges kulcsa, így minden járműhöz egyedi. Típusa: szám (int), nem lehet null.
- **vin:** A jármű alvázszáma, adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), nem lehet null, egyedi.
- **licensePlate:** A jármű rendszámablaja, adminisztrációs okokból tárolva. Típusa: szöveg (string / varchar), nem lehet null, egyedi.
- **manufacturer:** A jármű gyártója. Típusa: szöveg (string / varchar), nem lehet null.
- **model:** A jármű típusa, modelle. Típusa: szöveg (string / varchar), nem lehet null.
- **year:** A jármű gyártásának éve. Típusa: szám (int), nem lehet null.
- **description:** A jármű, tulajdonos általi, leírása. Típusa: szöveg (string / varchar), nem lehet null, de lehet üres.
- **odometerReading:** A jármű kilométerórájának állása. Típusa: szám (int), nem lehet null.
- **horsepower:** A jármű teljesítménye lóerőben. Típusa: szám (int), nem lehet null.
- **avgFuelConsumption:** A jármű átlag üzemanyagfogyasztása 100 km-en. Típusa: törtszám (double), nem lehet null.
- **fuelType:** A jármű üzemanyagának típusa (pl.: benzin, dízel, elektromos). Típusa: szöveg (string / varchar), nem lehet null.
- **insuranceNumber:** A jármű kötelező felelősségbiztosításának azonosítószáma. Típusa: szöveg (string / varchar), nem lehet null.
- **transmission:** A jármű váltójának típusa (pl.: manuális, automata). Típusa: szöveg (string / varchar), nem lehet null.

A leképezés után létrejövő idegen kulcs(ok):

- **ownerId:** A jármű tulajdonosának felhasználóazonosító száma, idegen kulcs. Típusa: szám (int), nem lehet null. A *users* táblára mutat.

Járműelérhetőség (*vehicleavailabilities*)

A jármű elérhetőségeit, bérlehetőségi időszakait tartalmazó egyedtípus/tábla.

Attribútumai/mezői:

- **id:** A backenden és az adatbázisban az elérhetőséget azonosító szám, a létrejövő táblában az elsődleges kulcs. Típusa: szám (int), nem lehet null.
- **availabilityId:** Egy jármű kontextusában az elérhetőséget azonosító szám. Típusa: szám (int), nem lehet null.
- **start:** Az adott elérhetőség kezdeti időpontja. Típusa: időpont (datetime, UTC), nem lehet null.
- **end:** Az adott elérhetőség végének időpontja. Típusa: időpont (datetime, UTC), nem lehet null.
- **hourlyRate:** A bérlehetőségi időszakban az óradíj. Típusa: szám (int), nem lehet null.

A leképezés után létrejövő idegen kulcs(ok):

- **vehicleId:** A járművet azonosító idegen kulcs, amelyhez az elérhetőség tartozik. Típusa: szám (int), nem lehet null. A *vehicles* táblára mutat.

Járműkép (*vehicleimages*)

A járműhöz tartozó képek elérési útját tartalmazó egyedtípus/tábla.

Attribútumai/mezői:

- **id:** A backenden és az adatbázisban a képet azonosító szám, a létrejövő táblában az elsődleges kulcs. Típusa: szám (int), nem lehet null.
- **imageId:** Egy jármű kontextusában a képet azonosító szám. Típusa: szám (int), nem lehet null.
- **sortIndex:** A járműhöz tartozó adott kép sorszáma. Típusa: szám (int), nem lehet null.
- **path:** A járműhöz tartozó kép elérési útja. Típusa: szöveg (string / varchar), nem lehet null.

A leképezés után létrejövő idegen kulcs(ok):

- **vehicleId:** A járművet azonosító idegen kulcs, amelyhez a kép tartozik. Típusa: szám (int), nem lehet null. A *vehicles* táblára mutat.

Bérlés (rentals)

A bérlés adatait és állapotát tároló egyed típus. A rekordok funkcionálhatnak bérlési ajánlatként és bérlésként is.

Attribútumai/mezői:

- **id:** A rendszerben a bérlés azonosítószáma, a létrejövő tábla elsődleges kulcsa, így minden bérléshez egyedi. Típusa: szám (int), nem lehet null.
- **start:** A bérlés kezdete. Típusa: időpont (datetime, UTC), nem lehet null.
- **end:** A bérlés vége. Típusa: időpont (datetime, UTC), nem lehet null.
- **rentalPrice:** A bérlés ára, a járműelérhetőségek óradíjai alapján kiszámolva. Típusa: szám (int), nem lehet null.
- **status:** A bérlés státusza. Típusa: enum (adatbázisban int), nem lehet null.

Lehetséges értékei:

- renterOffer (0): A bérlő bérlési ajánlata.
 - ownerOffer (1): A bérbeadó ellenajánlata.
 - offerAccepted (2): Az egyik fél elfogadta a másik ajánlatát, a bérlés viszont még nem kezdődött meg.
 - renterPickupAccepted (3): A bérlő megerősítette a jármű átvételét.
 - ownerPickupAccepted (4): A bérbeadó megerősítette a jármű átvételét.
 - active (5): A bérlés aktív, folyamatban van, amikor mindkét fél megerősítette az átvételt.
 - renterFinishAccepted (6): A bérlő megerősítette a jármű visszahozatalát.
 - ownerFinishAccepted (7): A bérbeadó megerősítette a jármű visszahozatalát.
 - finished (8): A bérlés befejeződött miután mindkét fél megerősítette a jármű visszahozatalát.
 - cancelled (9): Valamelyik fél lemondta a bérlést.
- **pickupLocation:** A felek által megbeszélt átvételi hely címe. Típusa: szöveg (string / varchar), nem lehet null.
 - **renterRating:** A bérlő értékelése a bérbeadóról. Típusa: törtszám (double), lehet null.
 - **ownerRating:** A bérbeadó értékelése a bérlőről. Típusa: törtszám (double), lehet null.

A leképezés után létrejövő idegen kulcs(ok):

- ***renterId***: A járműt bérlő felhasználó azonosítószáma, idegen kulcs. Típusa: szám (int), nem lehet null. A *users* táblára mutat.
- ***vehicleId***: A jármű azonosítószáma, amelyre a bérlet vonatkozik, idegen kulcs. Típusa: szám (int), nem lehet null. A *vehicles* táblára mutat.

Üzenet (*messages*)

A felhasználók között, egy bérlet kontextusában küldött, üzenetek adatait tároló egyed típus/tábla.

Attribútumai/mezői:

- **id**: A rendszerben az üzenetet azonosító szám, a létrejövő táblában elsődleges kulcs. Típusa: szám (int), nem lehet null.
- ***content***: Az üzenet tartalma, vagy amennyiben az üzenet egy kép, a kép elérési útja. Típusa: szöveg (string / varchar), nem lehet null.
- ***isImage***: Megadja, hogy az üzenet egy kép-e. Típusa: logikai (bool / tinyint), nem lehet null, alapvetően hamis.
- ***timeSent***: Az üzenet elküldésének időpontja. Típusa: időpont (datetime, UTC), nem lehet null.
- ***isComplaint***: Megadja, hogy az üzenet egyben jelentés-e az adminisztrátorok felé. Típusa: logikai (bool / tinyint), nem lehet null, alapvetően hamis.

A leképezés után létrejövő idegen kulcs(ok):

- ***rentalId***: A bérlet azonosítószáma, amelynek kontextusában az üzenet el lett küldve, idegen kulcs. Típusa: szám (int), nem lehet null. A *rentals* táblára mutat.
- ***senderId***: A felhasználó azonosítószáma, aki az üzenetet elküldte, idegen kulcs. Típusa: szám (int), nem lehet null. A *users* táblára mutat.

Kapcsolatok

Az egyed típusok közötti kapcsolatok az alábbi módon épülnek fel.

Felhasználó – Jármű (1:N, parciális-totális)

Nem minden felhasználóhoz tartozik jármű, így a felhasználó oldaláról a kapcsolat parciális, viszont minden járműnek van tulajdonos felhasználója, így a jármű oldaláról pedig a kapcsolat totális.

Egy felhasználó több járművet is birtokolhat és bérbe adhat a rendszeren keresztül, viszont egy járműnek csak egy tulajdonosa van a rendszerben, így ez egy 1:N kapcsolat.

Felhasználó – Értesítés (1:N, parciális-totális)

Nem minden felhasználó kapott értesítést, így a felhasználó oldaláról a kapcsolat parciális, viszont minden értesítésnek van címzett felhasználója, így az értesítés oldaláról pedig a kapcsolat totális.

Egy felhasználó több értesítést is kaphat, egy értesítést viszont csak egy felhasználó kap, így ez egy 1:N kapcsolat.

Felhasználó – Token (1:N, parciális-totális)

Nem minden felhasználó kér jelszóvisszaállító vagy más kódot, így a felhasználó oldaláról a kapcsolat parciális, viszont minden token egy felhasználó kér, így a token oldaláról a kapcsolat pedig totális.

Egy felhasználó több token is kérhet, egy token viszont csak egy felhasználóra vonatkozik, így ez egy 1:N kapcsolat.

Felhasználó – Bérlés (1:N, parciális-totális)

Nem minden felhasználó kezd bérlést, van, aki csak bérbe ad, így a felhasználó oldaláról a kapcsolat parciális, viszont minden bérlésnek van bérlője, így a bérlés oldaláról a kapcsolat totális.

Egy felhasználó több bérlést is megkezdhet, egy bérlésnek viszont csak egy bérlője lehet, így ez egy 1:N kapcsolat.

Felhasználó – Üzenet (1:N, parciális-totális)

Nem minden felhasználó küld üzenetet, így a felhasználó oldaláról a kapcsolat parciális, viszont minden üzenetnek van küldője, így az üzenet oldaláról ez egy totális kapcsolat.

Egy felhasználó több üzenetet is küldhet, egy üzenetnek viszont csak egy küldője van, így ez egy 1:N kapcsolat.

Jármű – Járműelérhetőség (1:N, parciális-totális)

Nem kötelező létrehozni elérhetőséget egy járműhöz, így a jármű oldaláról a kapcsolat parciális, viszont minden elérhetőség egy járműhöz tartozik, így az elérhetőség oldaláról a kapcsolat totális.

Egy járműhöz több elérhetőség is tartozhat, viszont egy elérhetőség csak egy járműhöz tartozik, így ez egy 1:N kapcsolat.

Jármű – Járműkép (1:N, parciális-totális)

Nem kötelező minden járműhöz képet csatolni, így a jármű oldaláról a kapcsolat parciális, viszont minden képhez tartozik egy jármű, így annak oldaláról a kapcsolat totális.

Egy járműhöz több kép is tartozhat, viszont egy kép csak egy járműhöz tartozik, így ez egy 1:N kapcsolat.

Jármű – Bérlés (1:N, parciális-totális)

Nem minden járművet béreltek ki, így a jármű oldaláról a kapcsolat parciális, viszont minden bérlés egy járműre vonatkozik, így a bérlés oldaláról a kapcsolat totális.

Egy járműhöz több bérlés is tartozhat, viszont egy bérlés csak egy járműre vonatkozik, így ez egy 1:N kapcsolat.

Bérlés – Üzenet (1:N, parciális-totális)

Nem minden bérlés kontextusában küldtek üzenetet, így a bérlés oldaláról a kapcsolat parciális, viszont minden üzenet egy bérlés kontextusában van küldve, így az üzenet oldaláról a kapcsolat totális.

Egy bérléshez több üzenet is tartozhat, viszont egy üzenet csak egy bérléshez tartozik, így ez egy 1:N kapcsolat.

Implementáció

A leképezés után létrejövő relációs adatbázis táblait MySQL-ben implementáltuk. Emellett még hozzáadtunk periodikusan lefutó eseményeket, melyek törlik a régi járműelérhetőségeket és lejárt visszaállító kódokat.

Backend

A backend alapvetően 5 metóduskontrollerre, 4 saját szolgáltatásra, és egy válaszsűrőrendszerre oszlik fel.

Metóduskontrollerek

Autentikációs kontroller – AuthController

Az autentikációs kontroller metódusai felelnek a rendszerbe való bejelentkeztetésért, kijelentkeztetésért, regisztrációért, illetve a felhasználók jelszavainak visszaállításáért.

API végpontjai:

[/api/Auth/Login](#)

- POST metódus:
 - Bejelentkezteti a felhasználót a rendszerbe.
 - Paramétere: **LoginDTO** (A bejelentkezni kívánt felhasználó e-mail címe és jelszava)
 - Visszatérési értékei:
 - **401 Unauthorized**: Amennyiben nem található a megadott e-mail címmel felhasználó, vagy az adott felhasználóhoz nem illik a jelszó.
 - **500 Internal Server Error**: Amennyiben nem sikerült legenerálni a hozzáférési tokent.
 - **200 OK (JWT HttpOnly Cookieval)**: Amennyiben a megadott bejelentkezési adatok helyesek.

[/api/Auth/Logout](#)

- POST metódus:
 - Kijelentkezteti a felhasználót, azzal, hogy törli a JWT HttpOnly Cookie-t, amennyiben létezik.
 - Nincs paramétere.
 - Visszatérési értéke:
 - **200 OK**: Törölve a Cookie-t, ha van.

[/api/Auth/Register](#)

- POST metódus:
 - Regisztrálja a felhasználót a rendszerbe.
 - Paramétere: **UserDTO** (A felhasználó regisztrációs adatait tartalmazza)
 - Visszatérési értékei:
 - **400 BadRequest:** Amennyiben a megadott regisztrációs adatok hibásak. (pl.: rossz e-mail formátum, nem múlt el 18 éves a regisztrálandó személy, stb.)
 - **409 Conflict:** Amennyiben a megadott regisztrációs adatok (csak e-mail, telefonszám, személyi szám, illetve jogosítványszám) ütköznek más felhasználók adataival.
 - **500 Internal Server Error:** Amennyiben nem sikerült legenerálni a hozzáférési tokent.
 - **200 OK (JWT HttpOnly Cookieval):** Amennyiben a regisztráció sikeres volt.

[/api/Auth/ForgotPassword](#)

- POST metódus:
 - Jelszóvisszaállító kódot küld a felhasználó e-mail címére.
 - Paramétere: A **felhasználó e-mail címe** (queryben)
 - Visszatérési értékei:
 - **503 Service Unavailable:** Amennyiben az e-mail küldő szolgáltatás nem elérhető.
 - **200 OK:** Mivel nem akarjuk tudatni, hogy mely e-mailek vannak regisztrálva a rendszerbe, minden más esetben ezt adjuk vissza.

[/api/Auth/ResetPassword](#)

- POST metódus:
 - Átállítja a jelszót a megadottra, amennyiben a megadott visszaállító kód megtalálható az adatbázisban.
 - Paramétere: **PasswordResetDTO** (e-mail, jelszóvisszaállító kód, új jelszó)
 - Visszatérési értékei:
 - **200 OK:** Amennyiben a jelszó sikeresen megváltozott.
 - **400 BadRequest:** Minden más esetben, ezzel védve a felhasználók e-mail címét.

Felhasználó kontroller – UserController

A felhasználó kontroller metódusai felelnek a felhasználó adatainak lekéréséért, frissítéséért, valamint az értesítéseinek kezeléséért.

API végpontjai:

/api/User

- GET metódus:
 - Lekéri a bejelentkezett felhasználó adatait.
 - Nincs paramétere.
 - Visszatérési értékei:
 - **401 Unauthorized:** Amennyiben nincs bejelentkezett felhasználó.
 - **200 OK (Felhasználó adataival):** Amennyiben sikeresen le lett kérve a felhasználó.
 - (Elméletileg a 404 Not Found is lehetséges lenne, amennyiben a felhasználó megszűnik létezni, de a hozzáférési tokent még a böngésző tárolja, viszont gyakorlatban ez szinte lehetetlen.)

/api/User/{id}

- GET metódus:
 - Lekéri a megadott azonosítójú felhasználó adatait (szűrve, attól függően, hogy ki kéri le).
 - Paramétere: id (az URL-ben): A lekérni kívánt felhasználó azonosítója.
 - Visszatérési értékei:
 - **404 Not Found:** Ha nem található a megadott azonosítóval rendelkező felhasználó.
 - **200 OK (Felhasználó adataival):** Amennyiben sikeresen le lett kérve a felhasználó.

- PUT metódus:
 - Frissíti az adott felhasználó adatait.
 - Paramétereit:
 - **id** (az URL-ben): A frissíteni kívánt felhasználó azonosítója.
 - **UserModificationDTO**: A felhasználó frissített adatait tárolja, valamint elkéri a felhasználó jelszavát a változtatások megerősítéséhez (amennyiben az nem adminisztrátor).
 - Visszatérési értékei:
 - **400 BadRequest**: A frissített adatok hibásak (pl.: nem jó formátumú e-mail cím, stb.)
 - **401 Unauthorized**: Nincs bejelentkezve a felhasználó, így nyilvánvalóan nem módosíthatóak az adatok.
 - **403 Forbidden**: Más felhasználó adatait nem módosíthatja senki, csak adminisztrátorok.
 - **404 Not Found**: Nincs ilyen azonosítójú felhasználó.
 - **409 Conflict**: A frissített adatok ütköznek egy másik felhasználó adataival. (e-mail cím, telefonszám, személyi szám, jogosítványszám)
 - **200 OK**: Az adatok sikeresen módosultak.

[/api/User/{id}/Image](#)

- PUT metódus:
 - Módosítja a felhasználó profilképét.
 - Paramétereit:
 - **id** (az URL-ben): A felhasználó azonosítója.
 - **A képfájl.**
 - Visszatérési értékei:
 - **401 Unauthorized**: Nincs bejelentkezve a felhasználó.
 - **403 Forbidden**: A bejelentkezett felhasználó másik felhasználó profilképét próbálja módosítani, és nem adminisztrátor.
 - **404 Not Found**: Nincs ilyen azonosítójú felhasználó.
 - **500 Internal Server Error**: Nem lehetett elmenteni a képet.
 - **200 OK**: A profilkép sikeresen módosult.

[/api/User/{id}/Deposit](#)

- PUT metódus:
 - Feltölti a felhasználó egyenlegét.
 - Paraméterei:
 - **id** (az URL-ben): A felhasználó azonosítója.
 - Az **összeg**, amennyit a felhasználó fel kíván tölteni.
 - Visszatérési értékei:
 - **400 BadRequest:** Helytelen összeg lett megadva (0 vagy kevesebb).
 - **401 Unauthorized:** Nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó másik felhasználó egyenlegét próbálja feltölteni, és nem adminisztrátor.
 - **404 Not Found:** Nincs ilyen azonosítójú felhasználó.
 - **200 OK:** A felhasználó sikeresen feltöltötte az egyenleget az adott összeggel.

[/api/User/{id}/Notification](#)

- GET metódus:
 - Visszaadja a felhasználó értesítéseit.
 - Paramétere: **id** (az URL-ben): A felhasználó azonosítója.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó másik felhasználó értesítéseit próbálja lekérni és nem adminisztrátor.
 - **200 OK (Értesítések listájával):** Az értesítések sikeresen le lettek kérve.
- DELETE metódus:
 - Törli a felhasználó összes értesítését.
 - Paramétere: **id** (az URL-ben): A felhasználó azonosítója.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó másik felhasználó értesítéseit próbálja törölni és nem adminisztrátor.
 - **204 NoContent:** Az értesítések sikeresen törölve lettek.

/api/User/{userId}/Notification/{notificationId}

- GET módszer:
 - Lekéri az adott értesítés adatait.
 - Paraméterei:
 - **userId** (az URL-ben): A felhasználó azonosítója.
 - **notificationId** (az URL-ben): Az értesítés azonosítója a felhasználó kontextusában.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó másik felhasználó értesítéséhez próbál hozzáférni és nem adminisztrátor.
 - **404 Not Found:** Nincs ilyen azonosítójú felhasználó vagy a felhasználón belül ilyen azonosítójú értesítés.
 - **200 OK (Értesítés adataival):** Sikeresen le lettek kérve az értesítés adatai.
- DELETE módszer:
 - Törli az adott értesítést.
 - Paraméterei:
 - **userId** (az URL-ben): A felhasználó azonosítója.
 - **notificationId** (az URL-ben): Az értesítés azonosítója a felhasználó kontextusában.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó másik felhasználó értesítését próbálja törölni és nem adminisztrátor.
 - **404 Not Found:** Nincs ilyen azonosítójú felhasználó vagy a felhasználón belül ilyen azonosítójú értesítés.
 - **204 NoContent:** Az értesítés sikeresen törölve lett.

Bérlés kontroller – RentalController

A bérlés kontroller metódusai felelnek a bérlési ajánlatok létrehozásáért, frissítéséért (ezzel elfogadásáért, megkezdéséért és lezárásáért is), valamint azok visszamondásáért/törléséért.

API végpontok:

/api/Rental

- GET metódus:
 - Lekéri a felhasználó bérléseit melyekben bérlőként szerepel.
 - Nincsen paramétere.
 - Visszatérési értékei:
 - **401 Unauthorized:** A felhasználó nincs bejelentkezve.
 - **200 OK (Bérlések listájával):** Amennyiben a felhasználó be van jelentkezve.
- POST metódus:
 - Létrehoz egy új bérlési ajánlatot.
 - Paramétere: **RentalDTO** (A bérlés alapvető adatait tartalmazza, felek, jármű stb.)
 - Visszatérési értékei:
 - **400 BadRequest:** Amennyiben rossz időintervallum lett megadva (pl.: a vége a kezdet előtt van), nincs megadva átvételi hely, a felhasználó fiókjában nincs beállítva jogosítványszám vagy az ajánlatot tevő felhasználónak nem elegendő az egyenlege.
 - **401 Unauthorized:** Amennyiben nincs bejelentkezve a felhasználó.
 - **403 Forbidden:** A felhasználó a saját járművére próbált ajánlatot tenni.
 - **404 NotFound:** Nincs olyan azonosítójú jármű, mint amelyet a bérlési ajánlatban a felhasználó megadott.
 - **409 Conflict:** A megadott időintervallumon nem lehetett bérlési ajánlatot létrehozni, mert nincsen létrehozva rá elérhetőségi időszak, vagy már van az időszakban bérlés, amellyel ütközik.
 - **201 Created (A létrejött bérlési ajánlattal):** Sikeresen létrejött a bérlési ajánlat.

/api/Rental/{id}

- GET metódus:
 - Visszaadja az adott azonosítójú bérlet adatait.
 - Paraméterei: **id** (URL-ben): a bérlet azonosítószáma
 - Visszatérési értékei:
 - **401 Unauthorized:** Amennyiben nincs bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó egy olyan bérletet kíván lekérni, amelyhez nincs hozzáférése (és nem adminisztrátor).
 - **404 NotFound:** Nincs a megadott azonosítóval rendelkező bérlet.
 - **200 (Bérlet adataival):** Sikeres lekérés esetén.
- PUT metódus:
 - Frissíti a bérletet. Ez lehet ellenajánlat, elfogadás, akár visszamondás, valami más státuszváltozás, vagy értékelés.
 - Paraméterei:
 - **id** (URL-ben): a bérlet azonosítója
 - **RentalDTO:** a változtatások a bérletben.
 - Visszatérési értékei:
 - **400 BadRequest:** Amennyiben nem adtunk meg átvételi helyet, az időintervallumot rosszul adtuk meg, vagy érvénytelen státuszt adtunk meg. (pl.: egyenleghiány, vagy a bérlet idő előtti megkezdése)
 - **401 Unauthorized:** Nincs bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem tartozik a bérlethez, így nem változtathat rajta (és nem adminisztrátor).
 - **404 NotFound:** Nincs ilyen azonosítóval rendelkező bérlet.
 - **409 Conflict:** A jármű nem bérelhető az adott időszakban, mert akkor már van más bérlet, vagy nincs elérhetőségi időszak.
 - **200 OK (Frissített bérlettel):** A bérlet sikeres frissítése esetén.
 - **204 NoContent:** A bérlet korai (ajánlat elfogadása előtti) státusz alatti visszamondás miatt törölve lett.

- DELETE metódus:
 - Visszamondja, vagy amennyiben a bérlet az ajánlattevő státuszban van, törli az adott bérletet.
 - Paramétere: id (URL-ben): a bérlet azonosítója
 - Visszatérési értékei:
 - **400 BadRequest:** A bérlet már aktív, befejezett vagy visszamondott.
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem tartozik a bérlethez, és nem adminisztrátor, így nem mondhatja vissza/törölheti azt.
 - **404 NotFound:** Nem található a megadott azonosítóval rendelkező bérlet.
 - **200 OK:** A bérlet sikeresen vissza lett mondva.
 - **204 NoContent:** A bérlet törölve lett, mivel még az ajánlattevő fázisban volt.

[/api/Rental/{id}/Message](#)

- GET metódus:
 - Lekéri a bérlet kontextusában küldött üzeneteket.
 - Paramétere: id (URL-ben): a bérlet azonosítója
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem tartozik a bérlethez, és nem adminisztrátor, így nem férhet hozzá.
 - **404 NotFound:** Nincs ilyen azonosítójú bérlet.
 - **200 OK (Üzenetekkel):** Sikeres lekérés esetén.

- POST metódus:
 - Üzenetet küld a bérlet kontextusában.
 - Paraméterei:
 - **id** (URL-ben): a bérlet azonosítója
 - **MessageSendDTO** (az üzenet tartalmát tartalmazza, illetve egy logikai értéket, ami jelöli, hogy ez az üzenet egy jelentés-e az adminisztrátoroknak)
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó (aki nem adminisztrátor) olyan bérlet kontextusában próbál üzenetet küldeni, amelyhez nem tartozik.
 - **404 Not Found:** Nincsen a megadott azonosítóval ellátott bérlet.
 - **201 Created (Az üzenettel):** Sikeresen el lett küldve az üzenet.

[/api/Rental/Owned](#)

- GET metódus:
 - Visszaadja a felhasználó azon bérleteit, amelyekben bérbeadóként szerepel.
 - Nincsen paramétere.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **200 OK (Bérletekkel):** Sikeresen le lettek kérve a bérletek.

Jármű kontroller – *VehicleController*

A jármű kontroller metódusai felelnek a járművek kereséséért, hozzáadásáért, frissítéséért, illetve azokhoz elérhetőségek és képek hozzáadásáért, módosításáért és törléséért.

API végpontjai:

[/api/Vehicle](#)

- GET metódus:
 - Visszaadja a (legalább egy elérhetőséggel rendelkező) járműveket szűrve a megadott értékek alapján.
 - Paraméterei:
 - **manufacturer (opcionális, string):** A keresett jármű márkája.
 - **model (opcionális, string):** A keresett jármű típusa/modellje.
 - **year (opcionális, int):** A keresett jármű gyártási éve.
 - **settlement (opcionális, string):** A település, ahol a járművet bérelni kívánjuk.
 - **fuelType (opcionális, string):** A jármű üzemanyagának típusa.
 - **transmission (opcionális, string):** A jármű sebességváltójának típusa.
 - **minRate (opcionális, int):** A legkisebb óradíjú jármű, amit a felhasználó bérelne.
 - **maxRate (opcionális, int):** A legnagyobb óradíjú jármű, amit a felhasználó bérelne.
 - **minPrice (opcionális, int):** A legkisebb teljes ár, amit a felhasználó kifizetne egy bérlésért.
 - **maxPrice (opcionális, int):** A legnagyobb teljes ár, amit a felhasználó kifizetne egy bérlésért.
 - **showOwned (opcionális, bool):** Mutassa-e a rendszer a saját járműveket.
 - Visszatérési értékei:
 - **200 OK (Járművekkel):** Sikeres lekérés esetén.

- POST metódus:
 - Új jármű hozzáadása.
 - Paraméterei: **VehicleDTO** (a jármű adatait tartalmazó objektum)
 - Visszatérési értékei:
 - **400 BadRequest:** A jármű adatai formailag hibásak (pl.: rendszám, alvázszám)
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **409 Conflict:** Már van a megadott azonosító adatok valamelyikével jármű a rendszerben.
 - **201 Created (Jármű adatokkal):** A járművet sikeresen létrehozták a rendszerben.

[/api/Vehicle/{id}](#)

- GET metódus:
 - Lekéri a megadott azonosítójú jármű adatait. (szűrve a bejelentkezett felhasználó szerint)
 - Paraméterei:
 - **id** (URL-ben): a jármű azonosítója
 - **rentalStart (opcionális, DateTime):** Árajánlat kalkulálása esetén a bérlet kezdeti dátuma.
 - **rentalEnd (opcionális, DateTime):** Árajánlat kalkulálása esetén a bérlet végének dátuma.
 - Visszatérési értékei:
 - **404 NotFound:** Nem található az adott azonosítóval rendelkező jármű.
 - **200 OK (Járművel):** Sikeres lekérés esetén.

- PUT metódus:
 - Frissíti a jármű adatait.
 - Paramétereit:
 - **id** (URL-ben): a jármű azonosítója
 - **VehicleDTO** (a jármű frissített adatait tartalmazó objektum)
 - Visszatérési értékei:
 - **400 BadRequest:** Formailag hibásak a megadott adatok.
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó, aki nem adminisztrátor, nem tulajdonosa a járműnek, így nem módosíthatja azt.
 - **404 NotFound:** Nincsen ilyen azonosítójú jármű.
 - **409 Conflict:** A frissített adatok ütköznek más jármű adataival.
 - **200 OK (Frissített járműadatokkal):** Sikeres lekérés esetén.

[/api/Vehicle/Owned](#)

- GET metódus:
 - Visszaadja a bejelentkezett felhasználó tulajdonában lévő járműveket.
 - Nincsenek paramétereit.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **200 OK (Járművekkel):** Sikeres lekérés esetén.

</api/Vehicle/{id}/Quote>

- GET metódus:
 - Lekér egy árajánlatot a megadott járműre, a megadott időszakban.
 - Paraméterei:
 - **id** (URL-ben): A jármű azonosítója.
 - **rentalStart (DateTime, UTC)**: Az ajánlat kezdeti időpontja.
 - **rentalEnd (DateTime, UTC)**: Az ajánlat végének időpontja.
 - Visszatérési értéke:
 - **400 BadRequest**: Nincs megadva kezdet vagy végső időpont, a végső időpont a kezdeti előtt van, vagy az ajánlat a múltra vonatkozik.
 - **404 NotFound**: Nincsen ilyen azonosítójú jármű.
 - **409 Conflict**: Ebben az időpontban már van bérlet, vagy nincsen elérhetőségi időszak a járműre.
 - **200 OK (Ajánlattal)**: Amennyiben van ajánlat az időszakra.

</api/Vehicle/{id}/Availability>

- GET metódus:
 - Lekéri a jármű elérhetőségi időszakait.
 - Paramétere: **id** (URL-ben): a jármű azonosítója
 - Visszatérési értéke:
 - **200 OK (Elérhetőségek listájával)**

- POST metódus:
 - Hozzáad egy elérhetőségi időszakot a járműhöz.
 - Paraméterei:
 - **id** (URL-ben): a jármű azonosítója
 - **VehicleAvailabilitiesDTO:** Az elérhetőség adatait tartalmazó objektum (kezdeti és végső dátum, óradíj)
 - Visszatérési értékei:
 - **400 BadRequest:** A végső dátum a kezdeti dátum előtt van, vagy az óradíj kisebb, mint 0.
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** Nem a bejelentkezett felhasználó tulajdonában van a jármű, amelyhez az elérhetőségi időszakot hozzá próbálja adni, és a bejelentkezett felhasználó nem adminisztrátor.
 - **409 Conflict:** A megadott időszakra már van megadva elérhetőségi időszak.
 - **201 Created (Elérhetőségi időszakkal):** Sikeres létrehozás esetén.

</api/Vehicle/{vehicleId}/Availability/{availabilityId}>

- GET metódus:
 - Lekéri az adott azonosítójú elérhetőségi időszak adatait.
 - Paraméterei:
 - **vehicleId** (URL-ben): a jármű azonosítója
 - **availabilityId** (URL-ben): az elérhetőség azonosítója a jármű kontextusában
 - Visszatérési értékei:
 - **404 NotFound:** Nincsen a megadott azonosítóval rendelkező jármű, vagy nincsen a járművön belül az adott azonosítóval elérhetőségi időszak.
 - **200 OK (Elérhetőségi időszakkal):** Sikeres lekérés esetén.

- PUT metódus:
 - Frissíti a megadott azonosítójú elérhetőségi időszakot.
 - Paraméterei:
 - **vehicleId** (URL-ben): a jármű azonosítója
 - **availabilityId** (URL-ben): az elérhetőség azonosítója a jármű kontextusában
 - **VehicleAvailabilitiesDTO:** A frissített elérhetőség adatait tartalmazó objektum (kezdeti és végső dátum, óradíj)
 - Visszatérési értékei:
 - **400 BadRequest:** A végső dátum a kezdeti dátum előtt van, vagy az óradíj kisebb, mint 0.
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** Nem a bejelentkezett felhasználó tulajdonában van a jármű, amely elérhetőségi időszakát frissíteni kívánja, és a bejelentkezett felhasználó nem adminisztrátor.
 - **404 NotFound:** Nincsen a megadott azonosítókkal elérhetőség.
 - **409 Conflict:** A megadott időszakra már van elérhetőség.
 - **200 OK (Frissített elérhetőségi időszakkal):** Sikeres frissítés.
- DELETE metódus:
 - Törli a megadott azonosítójú elérhetőségi időszakot.
 - Paraméterei:
 - **vehicleId** (URL-ben): a jármű azonosítója
 - **availabilityId** (URL-ben): az elérhetőség azonosítója a jármű kontextusában
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** Nem a bejelentkezett felhasználó tulajdonában van a jármű, amely elérhetőségi időszakát törölni kívánja, és a bejelentkezett felhasználó nem adminisztrátor.
 - **404 NotFound:** Nincsen a megadott azonosítókkal rendelkező elérhetőségi időszak.
 - **204 NoContent:** Sikeresen törölve lett az elérhetőségi időszak.

/api/Vehicle/{vehicleId}/Image

- GET metódus:
 - Visszaadja az adott járműhöz tartozó képeket.
 - Paramétere: **vehicleId** (URL-ben): a jármű azonosítója.
 - Visszatérési értékei:
 - **200 OK (Járműképekkel):** A képeket sikeresen lekértük.
- POST metódus:
 - Feltölt egy új képet a járműhöz.
 - Paramétere:
 - **vehicleId** (URL-ben): a jármű azonosítója.
 - **A képfájl.**
 - **sortIndex** (Query-ben, **opcionális**): A feltöltött kép sorszáma.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincsen bejelentkezett felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem a tulajdonosa a járműnek, amelyhez a képet hozzá kívánja adni, és nem adminisztrátor.
 - **404 NotFound:** Nincs a megadott azonosítóval jármű.
 - **500 Internal Server Error:** A fájlt nem lehetett elmenteni.
 - **201 Created (Járműképpel):** A kép sikeresen hozzá lett adva a járműhöz.

/api/Vehicle/{vehicleId}/Image/{imageId}

- GET módszer:
 - Lekéri az adott azonosítójú képet.
 - Paraméterei:
 - **vehicleId** (URL-ben): a jármű azonosítója.
 - **imageId** (URL-ben): a kép azonosítója a jármű kontextusában.
 - Visszatérési értékei:
 - **404 NotFound:** Nem található járműkép a megadott azonosítókkal.
 - **200 OK (Járműképpel):** Sikeres lekérés esetén.
- PUT módszer:
 - Frissíti a megadott járműkép sorszámát.
 - Paraméterei:
 - **vehicleId** (URL-ben): a jármű azonosítója.
 - **imageId** (URL-ben): a kép azonosítója a jármű kontextusában.
 - **sortIndex** (Query-ben): az új sorszám.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem a tulajdonosa a járműnek, amelyhez a kép tartozik, és nem adminisztrátor, így azt nem módosíthatja.
 - **404 NotFound:** Nincs a megadott azonosítókkal járműkép.
 - **200 OK (Frissített járműképpel):** Sikeres frissítés esetén.

- DELETE metódus:
 - Törli a megadott járműképet.
 - Paramétereit:
 - **vehicleId** (URL-ben): a jármű azonosítója.
 - **imageId** (URL-ben): a kép azonosítója a jármű kontextusában.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem a tulajdonosa a járműnek, amelyhez a kép tartozik, és nem adminisztrátor, így azt nem törölheti.
 - **404 Not Found:** Nincsen ilyen azonosítókkal járműkép.
 - **500 Internal Server Error:** Nem sikerült törölni a képet.
 - **204 NoContent:** A járműkép sikeresen törölve lett.

Adminisztrátor Kontroller – AdminController.cs

Az adminisztrátori kontroller metódusai visszaadják az adminisztrátorok számára fontos adatokat összegyűjtve.

API végpontjai:

/api/Admin/User

- GET metódus:
 - Lekéri az oldalon található összes felhasználót.
 - Nincs paramétere.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem adminisztrátor.
 - **200 OK (Felhasználók listájával)**

/api/Admin/Vehicle

- GET metódus:
 - Lekéri az oldalon található összes járművet.
 - Nincs paramétere.
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem adminisztrátor.
 - **200 OK (Járművek listájával)**

/api/Admin/Rental

- GET metódus:
 - Lekéri az oldalon található összes (aktív) bérletet.
 - Paramétere: **active** (Query-ben, **bool**): csak a nem befejezett vagy visszamondott bérleteket kéri le, ha igaz
 - Visszatérési értékei:
 - **401 Unauthorized:** Nincs bejelentkezve felhasználó.
 - **403 Forbidden:** A bejelentkezett felhasználó nem adminisztrátor.
 - **200 OK (Bérlések listájával)**

Szolgáltatások

A backend saját szolgáltatásokkal végez el olyan funkciókat, amelyek több metóduson is átnyúlnak, ezek a következők:

Autentikációs szolgáltatás – AuthService

Jelszavak

Az AuthService kezeli valójában a felhasználók jelszavainak titkosítását, ehhez PBKDF2 hash algoritmust használ, egy saját maga által generált sóval/salttal, valamint ez a szolgáltatás végzi a jelszó hashek összehasonlítását is.

Hozzáférési token

Ez hozza létre azt a hozzáférési tokenet is, amit bejelentkezéskor/regisztrációkor visszaadunk, és ami igazolja a bejelentkezett felhasználó kilétét. A token maga egy JSON Web Token (JWT) alapú HttpOnly Cookie, amelyet a böngésző tárol el, és küld vissza minden kéréssel, ameddig az le nem jár, vagy a kijelentkezés során azt nem töröltetjük a böngészővel.

A JWT Claimjeinek/Állításainak kiolvasását maga az ASP.NET JWT könyvtára megoldja, viszont ezen szolgáltatás *GetUser* metódusa kéri le az adatbázisból végül a bejelentkezett felhasználót a metódusokban.

Visszaállító kódok

A szolgáltatás felel még a jelszóvisszaállító kódok generálásáért, adatbázisba eltárolásáért, valamint mikor azt az oldalon valóban fel akarják használni, annak validációjáért, majd sikeres felhasználás után törléséért.

Email szolgáltatás – IEmailService – ResendEmailService

IEmailService

Az email küldő szolgáltatás alapvetően egy interfész, annak érdekében, hogy az email küldő klienseket/API-kat/szolgáltatásokat, a szoftver szempontjából viszonylag könnyen le lehessen cserélni.

ResendEmailService

Az email küldéshez mi a **Resend**⁴ szolgáltatását vettük igénybe, ami ingyen enged 3000 emailt küldeni havonta, valamint saját .NET könyvtárunk is van, amellyel viszonylag egyszerűen implementálható a funkció. A ResendEmailService, az IEmailService-ből származik le, ez implementálja annak metódusait a Resend könyvtár segítségével. Amennyiben az API kulcs, amely által működne a funkció nincs megadva, a végpontok, amelyek használatánál 503-as hibakódot adnak vissza, amely azt jelenti a szolgáltatás jelenleg nem elérhető.

Képerőforrás-kezelő szolgáltatás – IResourceService – LocalResourceService

IResourceService

A képfeltöltést kezelő szolgáltatás elsősorban egy interfész, szintén olyan indokból, hogy amennyiben esetleg más képtároló módot keresnénk a jövőben a váltás a szoftver szempontjából viszonylag könnyű legyen.

LocalResourceService

Jelenleg az IResourceService metódusait egyedül a LocalResourceService implementálja, amely a feltöltött képfájlokhoz saját GUID fájlneveket generál, majd azt a háttértárra menti a **wwwroot** (vagy opcionálisan más) mappába. A **wwwroot** egy olyan mappa, amelyből az ASP.NET backend statikus fájlokat oszt meg, a mi esetünkben a **/res** elérési út alatt, így a feltöltött képeket is.

⁴ <https://resend.com/>

A bérlések kezelése az egész projekt talán egyik legkomplexebb része, mivel az állapotváltozások miatt a bérlések állapotautomataként (state machine) kezelendők. Így ez egy külön szolgáltatást igényelt.

Állapotkezelés

Egy bérlés sokféle állapotban lehet:

- **renterOffer** (0): A bérlő bérlési ajánlata.
- **ownerOffer** (1): A bérbeadó ellenajánlata.
- **offerAccepted** (2): Az egyik fél elfogadta a másik ajánlatát, a bérlés viszont még nem kezdődött meg.
- **renterPickupAccepted** (3): A bérlő megerősítette a jármű átvételét.
- **ownerPickupAccepted** (4): A bérbeadó megerősítette a jármű átvételét.
- **active** (5): A bérlés aktív, folyamatban van, amikor mindkét fél megerősítette az átvételt.
- **renterFinishAccepted** (6): A bérlő megerősítette a jármű visszahozatalát.
- **ownerFinishAccepted** (7): A bérbeadó megerősítette a jármű visszahozatalát.
- **finished** (8): A bérlés befejeződött miután mindkét fél megerősítette a jármű visszahozatalát.
- **cancelled** (9): Valamelyik fél lemondta a bérlést.

Azonban egy mintát vehetünk észre a státuszok közt: úgymond szintekből állnak. Minden szintnek van két státusza, amely azt jelöli, hogy valamelyik fél elfogadta a következő lépésbe váltást, és van egy státusz, ami tulajdonképpen a következő nagy lépés/állapot. Nyilván csak akkor léphetünk a következő nagy állapotba, ha mindkét oldal elfogadta az abba váltást.

Például, ha én bérlő vagyok, akkor tudunk csak aktívra váltani, ha én elfogadom az átvételt, azzal, hogy aktívvá jelölöm a bérlést, és a bérlés státusza mikor ezt megteszem már **ownerPickupAccepted**, azaz a bérbeadó már elfogadta az átvételt. Amennyiben nem, akkor pedig miattam változik az állapot **renterPickupAccepted**-re, és működik ezután a folyamat ugyanígy a másik fél szempontjából.

A felhasználók a státusz frissítésekor mindig csak a végső (nagy) állapotokat (**offerAccepted**, **active**, **finished**) küldik el, amelyre végül váltani akarnak, kivéve az ajánlattevéskor, hiszen ekkor több ajánlat-ellenajánlat kör is bekövetkezhet.

Állapotkezelő algoritmus

Ez viszonylag könnyen lemodellezhető egy algoritmusként:

1. Keressük meg az első olyan „szintet”, amelyben a jelenlegi állapot van és amelynek a végső (nagy) státuszára próbálunk frissíteni.
 - a. Ha nincs ilyen, akkor vagy visszapróbáljuk mondani a bérletet (ez külön már le van kezelve, így tulajdonképpen lehetetlen), vagy ellenajánlatot akarunk tenni (ilyenkor a rendszer attól függően, hogy melyik fél a bejelentkezett felhasználó változtatja meg a státuszt, amennyiben a bérletben még nem lett elfogadva mindkét fél által egy ajánlat sem), vagy pedig nem érvényes státusz lett küldve.
 - b. Ha van ilyen, nézzük meg, hogy a másik fél már elfogadta-e a szint végső státuszába lépést.
 - i. Amennyiben igen, változtassuk az állapotot a szint végső státuszára.
 - ii. Amennyiben nem, változtassuk az állapotot a mi bérleti szerepünk általi elfogadásra.

Természetesen ettől függetlenül, még egy-egy státusz esetén kell különböző feladatokat végrehajtani, mint értesítésküldés vagy más bérleti ajánlatok (amelyek a megadott időszakra esnek) visszamondása.

Emellett státusztól függően módosítódnak a bérlet tulajdonságai:

- Ajánlattevéskor változtatható a kezdeti, és végső dátum és az átvételi hely (valamint változik az ár, habár azt az időintervallum és az elérhetőségi időszakok alapján számolja ki a rendszer)
- A bérlet végével a felhasználók értékelhetik a másikat.
- Az adminisztrátorok mindig változtathatnak (szinte) minden tulajdonságot (köztük a státuszt is).

Visszamondás

A visszamondás külön van kezelve, mivel eltér a többi státuszváltástól. Amennyiben a bérlet nem jutott túl az ajánlattevő időszakon, megpróbáljuk az egészet törölni, másképp csak a státuszt változtatjuk visszamondottra (cancelled-re), és mivel ekkor a bérlet ára már le lett vonva a bérletől és át lett adva a bérbeadónak, ezt visszafordítjuk. A visszamondásról vagy törlésről rendszerértesítést is kapnak a bérlet résztvevői.

Válaszszűrőrendszer – VisibilityFiltering

Vannak a rendszerben olyan osztályok/egyed típusok (névlegesen a User (felhasználók) és a Vehicle (járművek)), amelyek, habár alapvetően publikusak, azaz még a nem bejelentkezett felhasználók is megtekinthetik, azoknak jóval kevesebb adatot akarunk visszaadni róluk, mint mondjuk azok tulajdonosainak.

Ezért létrehoztunk egy válaszszűrőrendszert, amely 4 kategóriára osztja fel egy osztályon belüli tulajdonság láthatóságát (ez a **VisibilityLevel** enum):

- **Public:** mindenki által látható adatok (ez az alapvető a fel nem címkézett tulajdonságoknál)
- **InRelation:** csak azok által látható adatok, akikkel résztveszünk/résztvettünk bérlekben (csak ha a bérlet tovább jutott az ajánlattételéknél).
- **OwnerOnly:** csak a tulajdonos által látható adatok
- **AdminOnly:** csak az adminisztrátorok által látható adatok

A tulajdonságokat az osztályokon belül egy **VisibleTo** nevű attribútummal lehet felcímkézni, amely paraméterként a **VisibilityLevel** enum-ot kér be.

IFilterable

A szűrés csak olyan osztályokon történik, ahol meg van szabva milyen módon ellenőrizhető a bejelentkezett felhasználó láthatósági szintje. Az ilyen osztályoknak az IFilterable interfészből kell leszármazniuk, mivel ezen feltételeket egy *GetVisibilityConditionLambda* nevű függvényben kell implementálni.

JSON szerializáló

A szűrés maga a JSON szerializálás közben történik, rekurzívan, azaz a gyermekobjektumok (és folytatólagosan azok gyermekobjektumainak) tulajdonságai is szűrve vannak.

A szűréshez a JSON szerializáló egyik modulját a JsonTypeInfoResolver-t írjuk felül, amelynek feladata a megadott típus minden tulajdonságáról információkat összegyűjteni, köztük azt is, hogy kell-e szerializálni, vagy nem. Ezt a megállapító függvényt (ShouldSerialize) írjuk felül a *GetVisibilityConditionLambda* függvény eredmény lambdájával.

A saját TypeInfoResolverrel ellátott szerializálón keresztül kell szerializálnunk a megadott objektumot/listát, hogy a szűrés megtörténjen, ezt a *FilterSerialize* függvénnyel tehetjük meg.

A szerializáló a szűréstől függetlenül működik nem az IFilterable interfészből származó osztályokon is (viszont, ha van IFilterable gyermeke, azt már megszüri).

Frontend

Oldalak

A frontend több oldalra oszlik fel, ezek a *src/pages* mappa alatt találhatóak és (a következő) a „A frontend felület használatának bemutatása” részben mutatjuk be részletesen.

Komponensek

A frontend alapvetően több React komponensre oszlik fel az oldalakon belül is, ezeket az *src/components* mappában találhatjuk meg. A több oldalon használtakat pedig ezen belül a *common/* mappában találhatjuk meg.

Pár fontosabb komponens:

EnsureAuth

Ez a komponens nem látható (csak betöltéskor), azonban nagyon fontos dolga van, a React Router, *src/App.jsx*-ben a Routes alatt meghatározott oldalakon, lekéri, hogy van-e bejelentkezett felhasználó, amennyiben nincs átirányítja a felhasználót a bejelentkező felületre.

Navbar és NotificationMenu

A Navbar komponens határozza meg a navigációs sávot, amely szinte minden oldalon jelen van. Ez a sáv tartalmazza az oldalak közötti linkeket, a bejelentkezett felhasználó menüjét, amellyel eljuthat a számára releváns oldalakra vagy kijelentkezhet, és az értesítési menüt (NotificationMenu), amelyet megnyitva a felhasználó megtekintheti és törölheti a kapott rendszerértesítéseit.

CenteredLayout és PageLayout

Ezek a komponensek meghatározott keretet/kinézetet adnak az oldalaknak, amelyek használják.

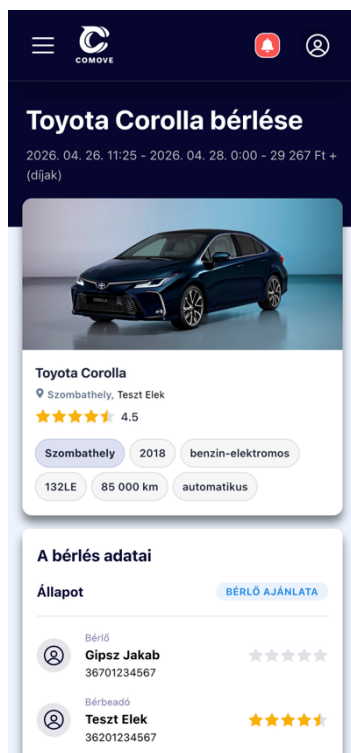
Scriptek

A `src/assets/scripts/` mappában találhatóak az oldal működéséhez fontos JavaScript fájlok, amelyek tartalmazzák az oldal konfigurációs beállításait (pl.: a backend URL-je), a bemeneti mezők ellenőrzéséhez használt reguláris kifejezéseket, és más általánosan használt függvényeket, mint például a `fetchAPI`, amely hibakezelten kér le adatokat a backend-től, a `checkOver18`, amely ellenőrzi, hogy a megadott (születési) dátum legalább 18 évvel ezelőtt volt-e, vagy a `trimForm`, amely leszedi az objektumok (legtöbbször Mantine form-ok) értékeinek elejéről és végéről a szóközöket.

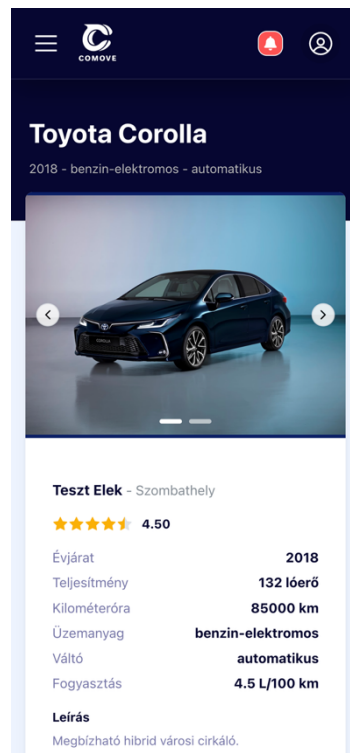
Emellett a `src/assets/scripts/hooks` mappában található pár saját React Hook, amelyek a bejelentkezett felhasználó adatainak lekéréséért, vagy kijelentkeztetéséért felelnek, illetve egy többször használt stílusbeállításokat meghatározó hook.

Reszponzivitás

Az oldal mobilon is kiválóan jelenik meg és működik, köszönhetően a Mantine könyvtár reszponzív komponenseinek, a beépített szélességi töréspontoknak, valamint az oldal stíluslapjainak.



Egy bérlet oldala mobilon (ábra 1)



Egy jármű oldala mobilon (ábra 2)



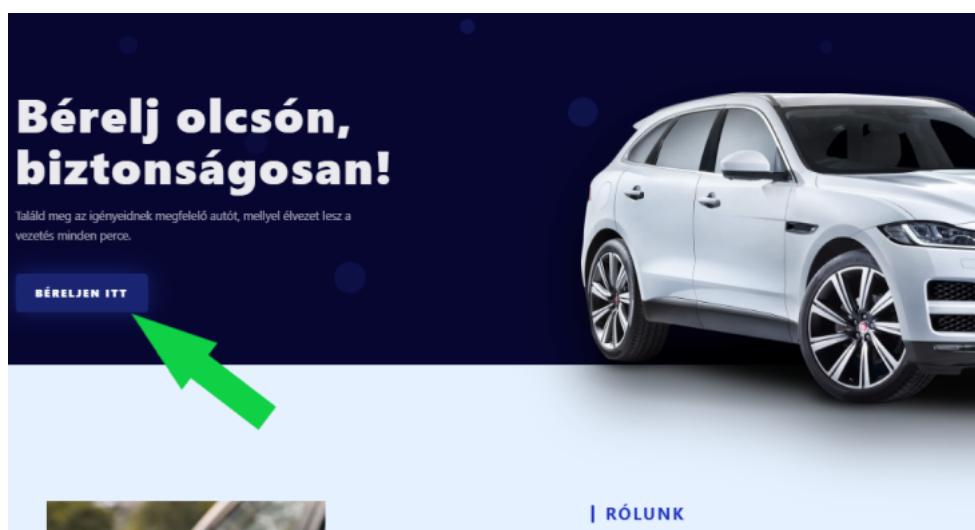
A főoldal mobilon (ábra 3)

A frontend felület használatának bemutatása

Ebben a fejezetben szeretnénk prezentálni a CoMove platform legfontosabb tulajdonságait a felhasználó szemszögéből. Lépésről-lépésre haladva magyarázzuk el az oldal működését, minden egyes részt és funkciót bemutatva egészen a regisztrációtól a bérlet lezárásáig.

Főoldal

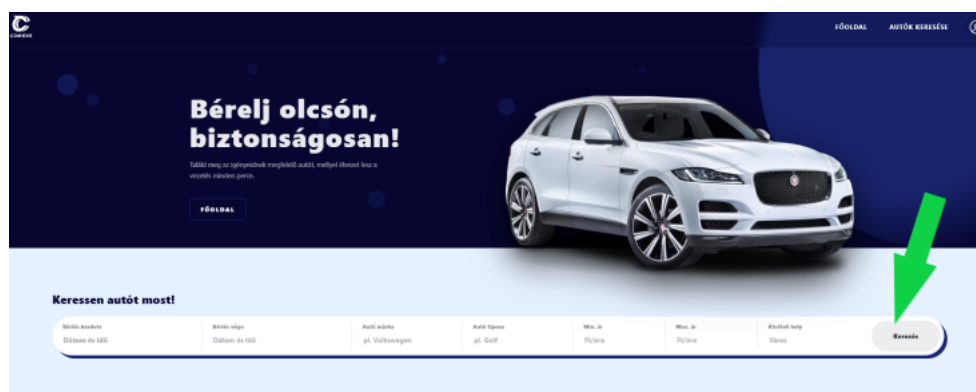
Mint minden egyes weboldalt amikor megnyitunk a főoldal tárul a szemünk elé. A főoldalon található „Rólunk” és „Miért válaszd a CoMove-ot?” részben röviden kifejtjük, hogy mi is a CoMove és mit nyújt a felhasználó számára. Az oldal lejjebb görgetésével megismerhetjük a CoMove mögött álló csapatot, a fejlesztőket, vagyis minket. Miután megismertük az oldal felépítését, fogjunk hozzá a bérlet megkezdéséhez.



Főoldal (ábra 4)

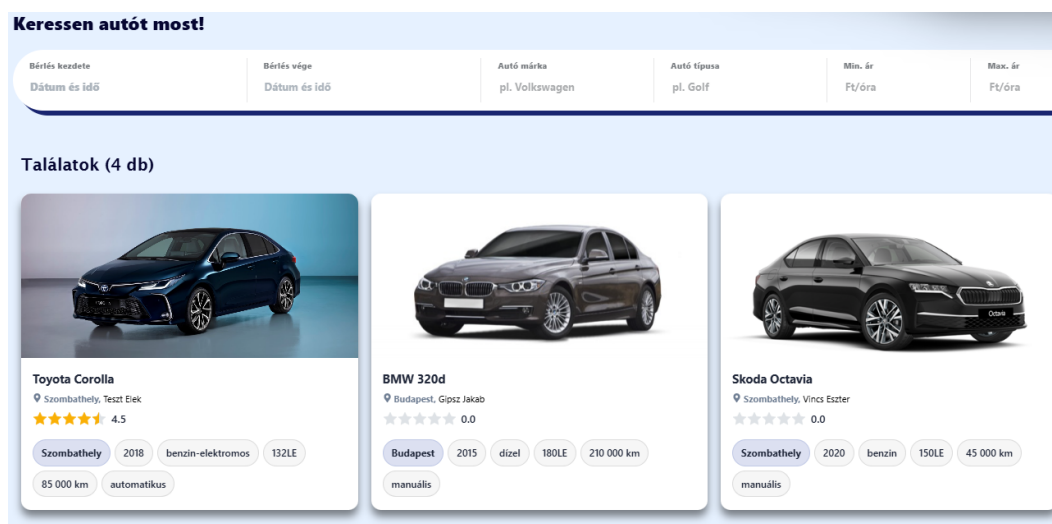
Az oldal tetején található fejlécben elhelyezett „Béreljen itt” gombra rákattintva az oldal átnavigál az autókereső oldalra, ahol már a bérelni kívánt autók listájából válogathatunk. Ezt az oldalt a Navigációs menüben lévő „Autók keresése” szövegre kattintva is el tudjuk érni.

Jármű kiválasztása



Keresősáv (ábra 5)

Az autó kereső oldal tartalmaz egy keresősávot, amelynek segítségével szűrni tudunk különböző adatok között, melyek szükségesek lehetnek a számunkra megfelelő jármű kiválasztásához. Ha szűrés nélkül szeretnénk autót keresni, akkor a „Keresés” gomb lenyomása után az összes meghirdetett bérbe vehető jármű megjelenik egy kis kártya formájában.



A keresés eredményeit megjelenítő járműkártyák (ábra 6)

A kártyák közt szabadon választhatunk és ha találtunk egy számunkra szimpatikus járművet, mely igényeinknek megfelel, a kártyát kiválasztva (vagyis arra kattintva) részletesebb leírást kapunk a járműről, valamint itt már meg kell adnunk, hogy mitől meddig szeretnénk kibérelni az autót. Ennél a pontnál a felhasználó, ha nincs bejelentkezve, nem tudja megkezdeni a bérletet, ezáltal nem tudja a bérletet tovább folytatni.

A jármű oldalán a bérlési ajánlat csak abban az esetben küldhető el, ha be vagyunk jelentkezve (ábra 7)

Regisztráció és Bejelentkezés

Ahhoz, hogy bérlést tudjunk leadni be kell jelentkezünk profilunkba. Amennyiben még nincs, úgy regisztrálnunk kell. A regisztráció három lépésből áll:

1. Személyes adatok megadása: teljes név, születési dátum, személyi igazolvány száma, és opcionálisan a jogosítványszám. A regisztrációhoz legalább 18 évesnek kell lenni.
2. Lakcím megadása: irányítószám, település és utca, házszám.
3. E-mail cím, telefonszám és jelszó megadása. A jelszónak legalább 8 karakter hosszúnak kell lennie, és tartalmaznia kell kis- és nagybetűt, valamint számot. A jelszót kétszer kell beírni a megerősítéshez.

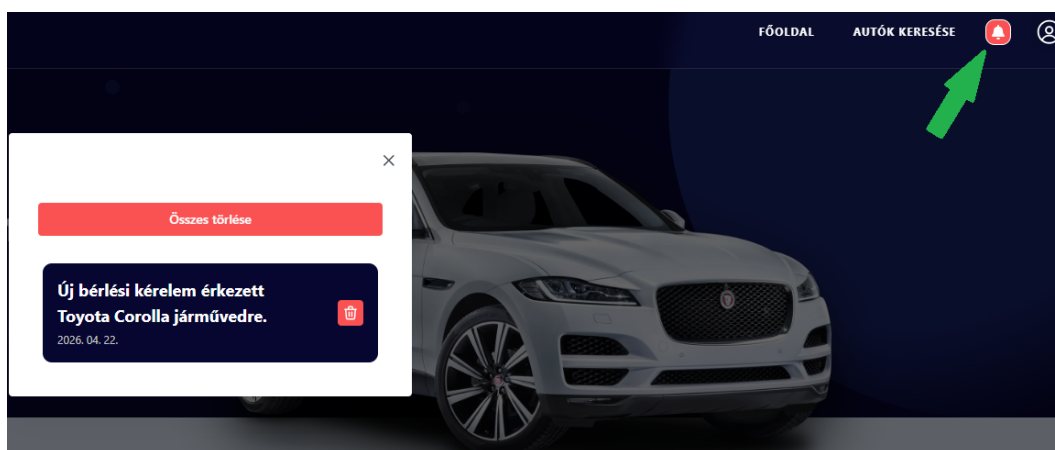
Minden lépésnél validáció fut, és hibaüzenet jelenik meg a nem megfelelően kitöltött mezők alatt. A "Regisztráció" gombra kattintva a rendszer létrehozza a fiókot, és a felhasználó automatikusan bejelentkezik. Ha már rendelkezünk fiókkal, akkor az oldalon a „Még nincs fiókod? Regisztrálj” szövegre kattintva tudunk egyszerűen bejelentkezni e-mail cím és jelszó megadásával. Elfelejtett jelszó esetén az „Elfelejtetted a jelszavad?” szövegre kattintva a megjelenő oldalon lehet új jelszót kérni, melyet egy e-mailbe lévő kóddal lehet megváltoztatni.

Regisztrációs felület (ábra 9)

Bejelentkező felület (ábra 8)

Bérlés folyamata

Miután sikeresen regisztráltunk és be vagyunk jelentkezve, valamint a kiválasztott autóhoz állítottunk be bérlés kezdeti és vég időpontot, illetve átvételi helyet, elküldhetünk egy bérlési ajánlatot. Ezután egy másik oldalra dob minket a platform, ahol láthatjuk az általunk kiválasztott autót, annak tulajdonságait, a bérlés adatait (ki a bérlő, ki a bérbeadó, kezdet és vég dátumokat, valamint a pénzügyeket melyek tartalmazzák a bérlés árát, a szolgáltatás díját és a teljes árat), és a saját ajánlatunkat. Itt módosítani tudjuk az eddig megadott időpontokat és átvételi helyszínt és vissza tudjuk mondani a bérlést. Találunk egy üzenőfelületet is, ahol a két fél könnyen tud valós időben információkat egyeztetni, és akár képeket küldeni a történekről. Az autó tulajdonosa (a bérbeadó) értesítést kap, hogy kérés érkezett autójának bérbevételére. Ezt a profilja melletti értesítés fület („kis csengő”) lenyitva tudja megtekinteni.



Az értesítések menü (ábra 10)

A profilt lenyitva a „Járműveim” ablakot kiválasztva az aktuális járműre rákattintva látjuk a járműre érkezett eddigi összes bérlést, a legrégebbitől a legfrissebbig.

Bérlések	
Bérlés 16 2026. 04. 23. - 2026. 04. 24. - 20 160 Ft	BÉRLŐ AJÁNLATA
Bérlés 15 2026. 04. 29. - 2026. 05. 01. - 44 520 Ft	ELFOGADOTT AJÁNLAT
Bérlés 5 2026. 03. 24. - 2026. 03. 26. - 41 160 Ft	LEMONDVA
Bérlés 1 2026. 01. 05. - 2026. 01. 05. - 15 750 Ft	BEFEJEZŐDÖTT

A jármű oldalán az arra vonatkozó bérlések listája (ábra 11)

A bérbeadónak a bérléssel kapcsolatba 3 opciója is van. A legegyszerűbb, amennyiben minden adat megfelel, elfogadhatja a bérlest. A második, ha számára mégsem alkalmas a bérbeadás, egy gombnyomással visszamondhatja azt. Végül, küldhet ellenajánlatot, amivel megváltoztathatja a bérlet időpontjait, vagy annak átvételi helyszínét. Módosítás esetén a bérlő értesítést kap minden információ változásról. Ha a bérlőnek nem felel meg, ő is küldhet egy ellenajánlatot, viszont, ha valamelyik oldal elfogadja a másik ajánlatát, a bérlet elfogadottá válik, és ekkor az ajánlat már rendes bérlet alakul.

The screenshot shows a form titled 'Ajánlat' (Offer). Below the title is a subtitle: 'Ajánlatot kaptál, alább megerősítheted, vagy ellenajánlatot tehetsz.' (You received an offer, below you can confirm it or make a counter-offer). The form contains three input fields: 'Bérlés kezdete' (Rental start) with a date and time picker set to '2026. 04. 23. 10:00', 'Bérlés vége' (Rental end) with a date and time picker set to '2026. 04. 24. 10:00', and 'Átvételi hely' (Pickup location) with a location picker set to 'Szombathely, Király utca 1'. At the bottom, there are three buttons: a green 'Elfogadás' (Accept) button with a checkmark icon, a dark blue 'Ellenajánlat küldése' (Send counter-offer) button, and a red 'Visszamondás' (Cancel) button with an 'X' icon.

A bérleti ajánlatokat kezelő felület (ábra 12)

Ezután már csak egy lépés van hátra, megvárni a bérlet kezdeti időpontját, hogy átvehessük az általunk választott autót. Miután átvettük a járművet, és a lefoglalt időintervallum után visszavittük tulajdonosához (mindezt a szoftverben mindkét oldalról megerősítve) a „csúszka” segítségével lezártnak tekinthetjük a bérletet.

The screenshot shows a confirmation screen titled 'Elfogadott ajánlat' (Accepted offer). It displays the rental details: 'Bérlés kezdete' (Rental start) as '2026. 04. 23. 10:00', 'Bérlés vége' (Rental end) as '2026. 04. 24. 10:00', and 'Átvételi hely' (Pickup location) as 'Szombathely, Király utca 1'. Below this, there is a progress bar with three steps: 1. 'Ajánlat elfogadva' (Offer accepted) with a green checkmark icon, 2. 'Átvétel megerősítve' (Pickup confirmed) with a green circle containing the number 2, and 3. 'Visszavétel megerősítve' (Return confirmed) with a grey circle containing the number 3. At the bottom, there are two buttons: a green 'Átvétel megerősítése' (Confirm pickup) button with a checkmark icon, and a red 'Visszamondás' (Cancel) button with an 'X' icon.

A bérlet állapotát megjelenítő felület, amelyen keresztül a következő állapotba is léphetünk (ábra 13)

Profil

A profil lenyitáskor láthatjuk, hogy több opció közül választhatunk, kizárólag csak akkor, ha be vagyunk jelentkezve. Amennyiben nem vagyunk bejelentkezve úgy a Regisztráció és Bejelentkezés lehetősége áll fent.



A profil menü (ábra 14)

Beállítások

A „Fiókbeállítások” oldalon a felhasználó a regisztráció során megadott adatait tudja egyszerűen módosítani. Új jelszó beállítása is lehetséges, amit a régi jelszó megadásával lehet csak lecserélni. Ezen kívül a felhasználó profilképet is tud beállítani, valamint törölni, illetve képes egyenlegét feltölteni, ami elengedhetetlen a bérléshez.

A screenshot of a web-based account settings form. The form is titled 'Egyenleg' and shows a balance of '62 045 Ft'. On the left, there is a profile picture placeholder with an upload icon and a delete icon. The right side of the form contains various input fields for personal and account information. The fields are: 'Név' (Teszt Elek), 'E-mail cím' (tesztelek@teszt.hu), 'Telefonszám' (36201234567), 'Születési dátum' (2004. 04. 18.), 'Személyi igazolvány szám' (123456AA), 'Jogosítványszám' (AA123456), 'Irányítószám' (9700), 'Település' (Szombathely), 'Utca, házszám' (Zrínyi Ilona utca 12.), 'Új jelszó' (with a strength indicator), and 'Régi jelszó' (with a toggle for visibility). A 'Mentés' button is located at the bottom right of the form.

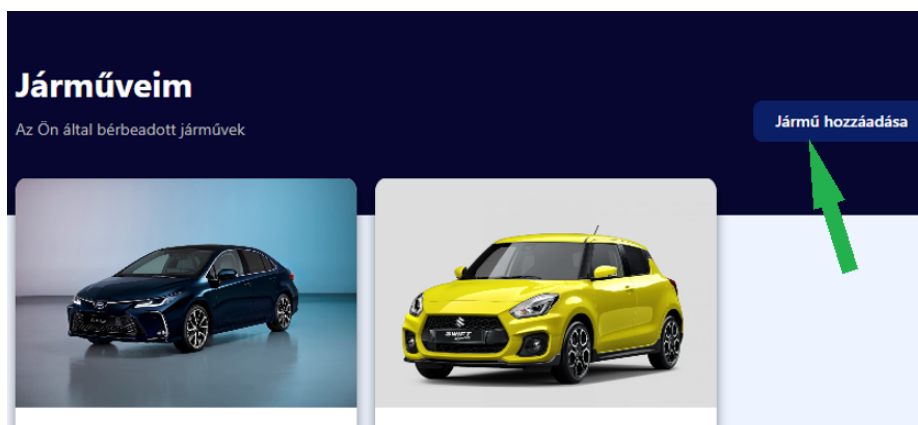
Fiókbeállítások (ábra 15)

Bérléseim

Itt láthatjuk az összes általunk indított bérléseket, ketté szedve archív és aktív részekre, amelyekre kattintva áttudunk lépni az ahhoz a bérléshez tartozó bérlési felületre.

Járműveim

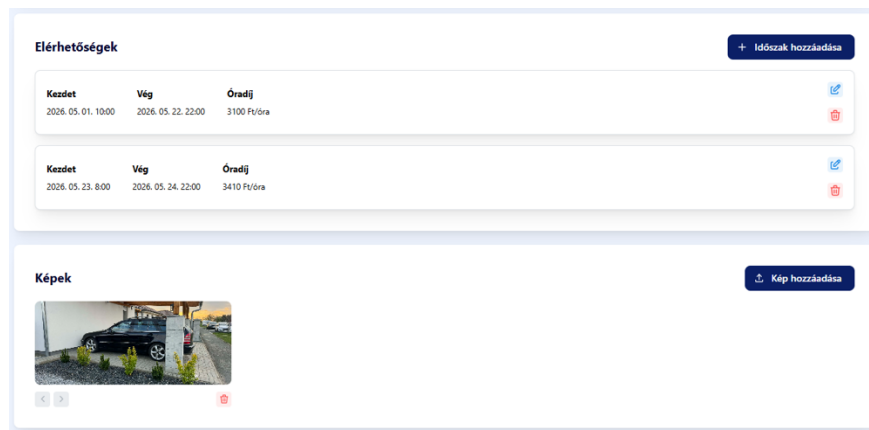
Fontos, hogy ha egy felhasználónk több magángépjárművel rendelkezik otthon és szeretné mindazon járműveit elérhetővé tenni az oldalon, mi erre lehetőséget nyújtunk.



Járműveim felület és jármű hozzáadása gomb (ábra 16)

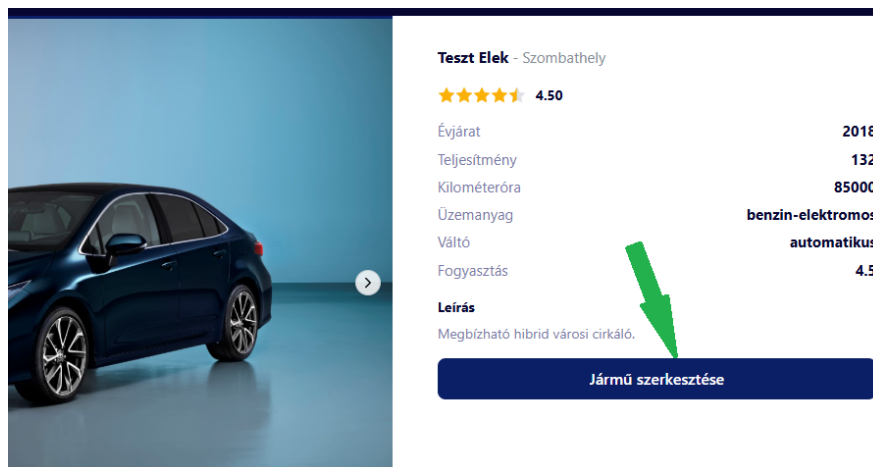
A „Jármű hozzáadása” választásakor egy űrlaphoz hasonló oldalra navigálódunk, ahol a járműhöz tartozó fontos információkkal kell kitöltenünk a mezőket. Kitöltés során megadjuk a gyártót és modellt, a jármű évjáratát, üzemanyag típusát, teljesítményt, aktuális kilométeróra állást, stb. Mentés után további két felületet tudunk kezelni. Az egyiken a jármű bérlésre való elérhetőségét tudjuk beállítani, a másikon pedig egy vagy akár több képet is tudunk csatolni gépjárművünkhöz, valamint beállíthatjuk a sorrendjüket. Amint ezzel megvagyunk, a jármű felkerült járműveink közé, illetve a bérlők a megadott időszakokban a járművet elérhetőnek látják a keresések során.

Jármű hozzáadása űrlap (ábra 17)



Az elérhetőség- és képkezelő felület (ábra 18)

A létező járműveink adatait könnyen meg tudjuk változtatni. A jármű kártyájára kattintva, majd a „Jármű szerkesztése” gombot lenyomva ugyanaz a felület jelenik meg, amelyen a jármű regisztrálását végeztük.



A jármű oldala és a jármű szerkesztése gomb, amellyel módosítani lehet a jármű adatait (ábra 19)

Adminisztrátori felület

Amennyiben egy felhasználó adminisztrátori szerepben van jelen az oldalon, saját autókat nem tölthet fel arra, és nem indíthat saját bérleteket sem, hiszen ezek összeférhetlenségeket teremtenének. Az adminisztrátorok egy központi felületről hozzáférhetnek az összes felhasználó, jármű és aktív bérlet adataihoz, illetve (nagyra) módosíthatják azokat.

Kijelentkezés

Ha már nem szeretnénk az oldalon végrehajtani semmilyen változtatást, és már nincs más teendők, akkor a kijelentkezéssel kiléphetünk profilunkból, amit követően a rendszer visszairányít bennünket a főoldalra.

Továbbifejlesztési lehetőségek

- **Jelentési/ügyfélszolgálati felület fejlesztése:** Habár létezik egy alapvető adminisztrátori felület, hasznos lenne, ha a felhasználók tudnának problémákat jelezni a bérlésekkel kapcsolatban, és az adminisztrátorok azokat valamiféle módon tudnák kezelni.
- **E-mail cím megerősítés és telefonszám ellenőrzés:** A jelszó visszaállítás e-mailen keresztül fontos funkció, hiszen így nem az oldal fenntartóinak kell módosítani a felhasználók jelszavait, hanem ezt automatizáltan megtehetik maguk. Az e-mail cím és telefonszám megerősítés azonban szintén fontos lehet, hiszen ezek egyrészt bot-szűrőként is funkcionálnak, másrészt pedig védik a felhasználók fiókjait.
- **Fizetésfeldolgozó szolgáltatás implementálása (pl.: SimplePay vagy Stripe):** Az oldalon jelenleg nincsen valós pénzkezelés, akárki akármennyi „pénzt” feltölthet. Egy fizetésfeldolgozó szolgáltatás implementálásával, valós pénzzel lehetne feltölteni az egyenleget, valamint kiutalni arról valós bankszámlára.
- **KYC szolgáltatás implementálása:** A Know Your Customer (KYC), azaz magyarul ügyfélismeret, egy nagyon fontos fejlesztés lenne a platformon, hiszen fontos megerősíteni, hogy valóban egy igazi emberrel kezdünk-e bérletet. Ezzel a szolgáltatással, a felhasználóknak nemcsak be kellene írni a személyazonosító okmányaik számát, hanem fotókkal, videókkal is igazolni személyazonosságukat.
- **Térkép integráció:** Valamiféle térkép szolgáltató API segíthetne az átadás/átvételi helyek kiválasztásában/vizualizálásában, felhasználók címeinek megerősítésében, és a járművek hely-alapú keresésében.
- **Szöveges értékelések:** Jelenleg a bérbeadók és bérlők 5 csillagból tudják egymást értékelni, amely hasznos, és azonnali képet ad, viszont kontextus nélkül sem a bérbeadó, sem a bérlő nem tanulhat belőle, hogy mit tehetne jobban a következő alkalommal.

Teszt dokumentáció

A backend rendszer végpontjainak metódusait **MSTest** egységtesztekkel teszteltük. Ezek az alábbi módon oszlanak meg.

TestHandler

A tesztek előkészítését és a tesztkörnyezet létrehozását egy *TestHandler.cs* nevű fájlban intézzük, ez mockolja az adatbázis kontextust, tesztadatokkal feltöltve, egy memóriában élő SQLite adatbázissal.

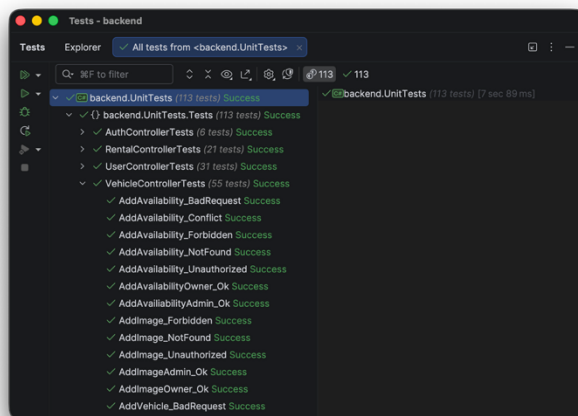
Emellett mockolja még a különböző szolgáltatásokat, például a *LocalResourceService* helyett *MockResourceService*-t hoz létre (így a tesztelt feltöltött fájlok nincsenek mentve valójában a háttértárra), illetve a *TimeProvider* helyett *FakeTimeProvider*-t hoz létre, így mi kontrollálhatjuk, hogy a tesztelés során a rendszer mit gondol jelen időnek.

Emellett a *TestHandler* végzi az aktuális bejelentkezett felhasználó mockolását is, így nem kell valójában hozzáférési tokeneket létrehozni, hanem azt beállíthatjuk a *Claims*-ben/Állításokban manuálisan a *SetAuthUser* metódusban.

Kontroller metódusok tesztelése

A kontrollerek metódusait az alábbi osztályokban teszteljük:

- **AuthControllerTests:** Az *AuthController* metódusait teszteli, a bejelentkezés, kijelentkezés különböző válaszait.
- **UserControllerTests:** A *UserController* metódusait teszteli, a felhasználói adatok lekérését, módosítását, az értesítések lekérését és törlését, figyeli a bejelentkezett felhasználó alapján a válaszsűrést is.



Sikeres tesztesetek (ábra 20)

- **VehicleControllerTests:** A *VehicleController* metódusait teszteli, a járművek adatainak, elérhetőségeinek, képeinek feltöltését, lekérését, módosítását és törlését teszteli, figyeli a bejelentkezett felhasználó alapján a válaszsűrést is.
- **RentalControllerTests:** A *RentalController* metódusait teszteli, ezáltal pedig az állapotkezelő rendszert is.

Stressztesztelés

Az egységteszt mellett teszteltük az API válaszidejét stresszteszteléssel. Ehhez a **Locust** nevű Python csomagot használtuk, amit a Python csomagkezelőjével lehet telepíteni, a pip-pel: *pip install locust*.

A stresszteszt elindításához először el kell indítanunk a backend-et **http** módban, másképp, mivel a locust nem ismeri fel a backend SSL tanúsítványát, minden mért adat hibás lesz. Emellett még fontos, hogy a backend-et debugging nélkül indítsuk, másképpen az jelentősen visszafogja a válaszidőket.

Miután a backend fut, lépünk be a *loadtest/* mappába és adjuk ki a *locust* parancsot, majd nyissuk meg a böngészőnkben a program által megadott oldalt, és kezdjük meg a tesztet.

Természetesen a stresszteszt korlátozva van abból a szempontból, hogy a backend lehet képes lenne párhuzamosan több kérést is kezelni, viszont az adatbázis nem biztos.

Fontos kiemelni emellett, hogy itt nem mockolt adatbázisról van szó, hanem a valódi adatbázissal való összekötés mellett mérjük az eredményeket. (Csak lekérünk adatokat, módosítani nem módosítunk semmit.)

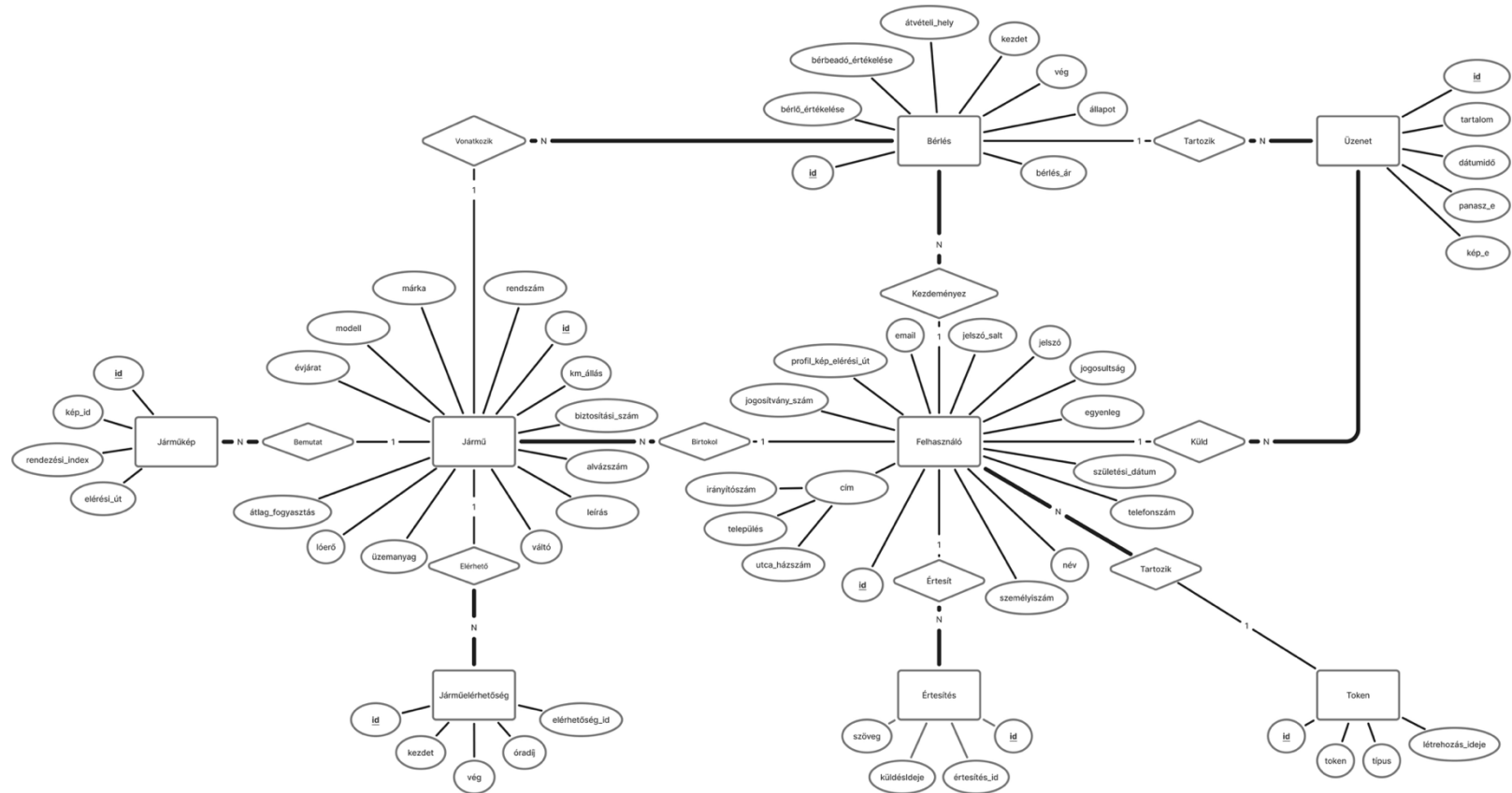
A mért adatok megmutatják a válaszidők átlagát, minimumát és maximumát, a kérések másodpercenkénti számát végpontonként és egyben is.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	/api/Auth/login	31612	0	109.36	16	2540	161.13	199.44	0
GET	/api/Rental	30360	0	101.25	19	3284	2	191.54	0
GET	/api/Rental/Owned	30547	0	101.12	21	3649	7051	192.72	0
GET	/api/User/1	9868	0	120.74	23	3306	5095	62.26	0
GET	/api/User/1/Notification	30846	0	98.2	21	3544	1080	194.61	0
GET	/api/User/2	10269	0	120.12	23	3370	679	64.79	0
GET	/api/User/3	10232	0	117.5	23	3310	597	64.55	0
GET	/api/Vehicle	30446	0	115.14	23	3702	3289	192.08	0
GET	/api/Vehicle/1	10292	0	113.01	22	3065	4967	64.93	0
GET	/api/Vehicle/1/Availability	30449	0	99.53	22	3312	1042	192.1	0
GET	/api/Vehicle/1/Image	10250	0	99.79	22	3555	246	64.67	0
GET	/api/Vehicle/2	10177	0	113.44	23	3007	1660	64.21	0
GET	/api/Vehicle/2/Image	10144	0	101.22	21	3570	124	64	0
GET	/api/Vehicle/3	9989	0	115.41	23	3233	1343	63.02	0
GET	/api/Vehicle/3/Image	10035	0	98.06	22	2917	124	63.31	0
GET	/api/Vehicle/Owned	30523	0	113.81	23	3601	8275	192.57	0
Aggregated		306039	0	107.14	16	3702	2575.19	1930.8	0

Stresszteszt eredmények 800 párhuzamos felhasználó mellett, 1930 kéréssel másodpercenként (ábra 21)

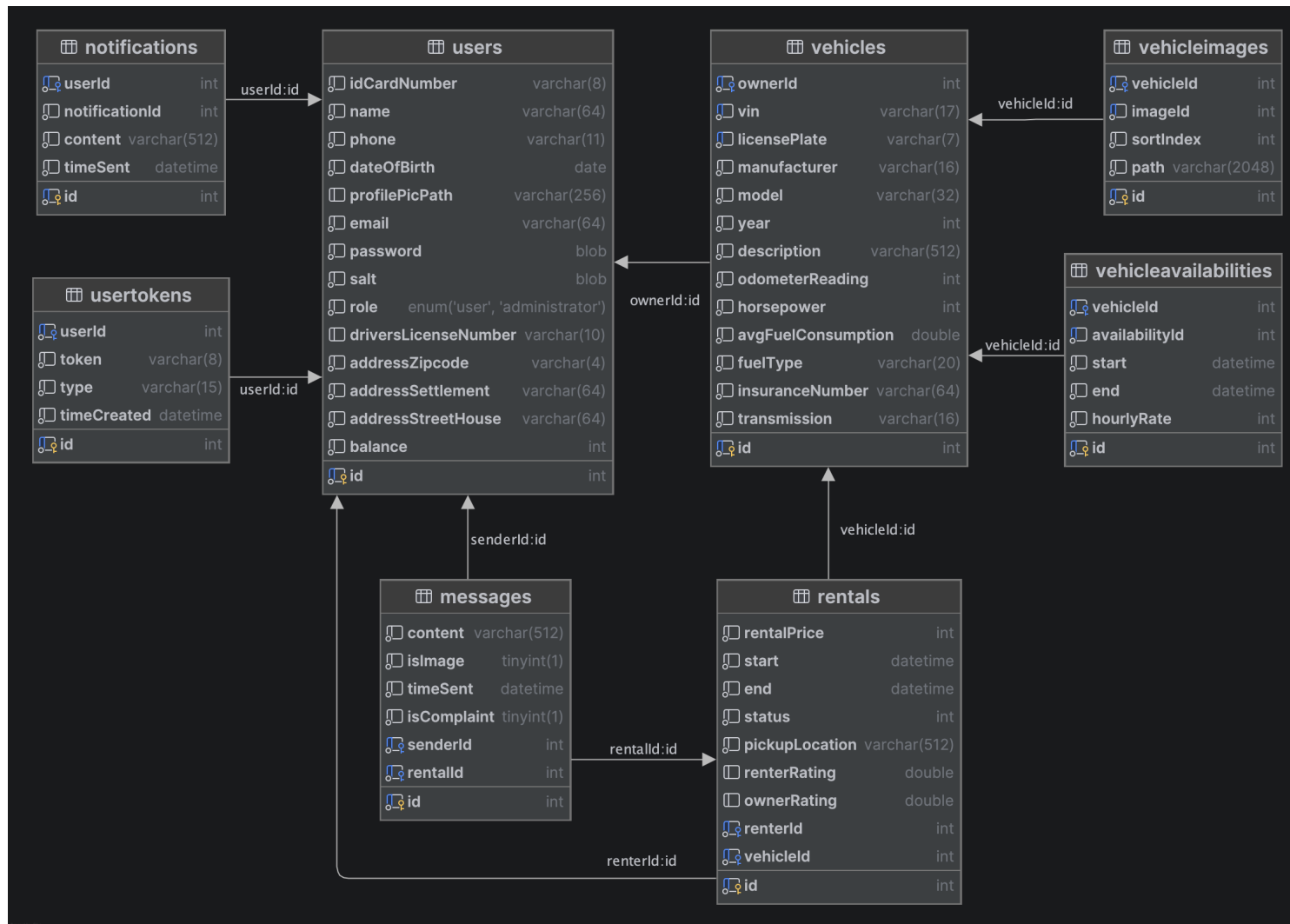
Mellékletek

ER-modell



ER-modell (ábra 22)

Relációs diagramm



Relációs diagramm (ábra 23)